JOB CLASSIFICATION USING MACHINE LEARNING ALGORITHMS

A Project Report Submitted to

Jawaharlal Nehru Technological University-Anantapur

In partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

K. PRASHANTH KUMAR REDDY	**-**	**18AK1A0592**
M.VAMSI KRISHNA	**-**	**19AK5A0504**
L. MANOJ KUMAR	**-**	**18AK1A0578**
S. MANOJ KUMAR	**-**	**18AK1A0579**

Under the guidance of

Mrs. B. Rupa Devi, MTech, (Ph.D.)

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

Venkatapuram(V), Karakambadi(Po), Renigunta(M), Tirupati-517520, A.P.

2018-2022

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

Venkatapuram(V), Karakambadi(Po), Renigunta(M), Tirupati-517520, A.P

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that this is a bonafide record of the Project Report entitled, **"JOB CLASSIFICATION USING MACHINE LEARNING ALGORITHMS"**, done by **Mr. K. Prashanth Kumar Reddy, REG NO.18AK1A0592,Mr.M.Vamsi Krishna, REG NO.19AK5A0504, Mr. L. Manoj Kumar, REG NO.18AK1A0578, Mr. S. Manoj Kumar, REG NO.18AK1A0579** Submitted to the faculty of Computer Science and Engineering, in partial fulfillment of the requirements for the Degree of **BACHELOR OF TECHNOLOGY** in **Computer Science and Engineering** from Jawaharlal Nehru Technological University, Ananthapuramu during 2018-2022.

Guide:
Mrs. B. Rupa Devi, MTech, (Ph.D.)
Assistant Professor,
Dept of CSE,
AITS, Tirupati.

Head of The Department
Mr. B. Ramana Reddy, MTech
Assistant Professor & HOD,
Dept of CSE,
AITS, Tirupati.

Date: _____

Place: Tirupati

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

Venkatapuram(V), Karakambadi(Po), Renigunta(M), Tirupati-517520, A.P

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We hereby declare that the project titled **"JOB CLASSIFICATION USING MACHINE LEARNING ALGORITHMS"** is a genuine project work carried out by us, in **B. Tech (Computer Science and Engineering)** degree course of **Jawaharlal Nehru Technology University, Ananthapuramu** and has not been submitted to any other course or university for the award of our degree by us.

K. PRASHANTH KUMAR REDDY - 18AK1A0592
M.VAMSI KRISHNA - 19AK5A0504
L. MANOJ KUMAR - 18AK1A0578
S. MANOJ KUMAR - 18AK1A0579

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We avail this opportunity to express our deep sense of gratitude and hearty thanks to **Mr. C. GANGI REDDY,** Hon'ble Secretary of AITS-Tirupati, for providing a congenial atmosphere and encouragement.

We show gratitude to **Dr. C. NADHAMUNI REDDY**, **Principal** for having provided all the facilitiesand support.

We would like to thank **Mr. B. RAMANA REDDY, Assistant Professor & HOD**, **Computer Science and Engineering** for encouragement at various levels of our Project.

We are thankful to our guide **Mrs. B. RUPA DEVI, Assistant Professor, Computer Science and Engineering** for her sustained inspiring guidance and cooperation throughout the process of this project. Her wise counsel and suggestions were invaluable.

We are thankful to **YOUNG MINDS** for providing an internship for our project.

We express our deep sense of gratitude and thanks to all the **Teaching** and **Non-Teaching Staff** of ourcollege who stood with us during the project and helped us to make it a successful venture.

We place our highest regards to our **Parents**, **Friends,** and **Well-wishers** who helped a lot in making the report on this project.

K. PRASHANTH KUMAR REDDY - 18AK1A0592
M.VAMSI KRISHNA - 19AK5A0504
L. MANOJ KUMAR - 18AK1A0578
S. MANOJ KUMAR - 18AK1A0579

CONTENTS

CHAPTER NO	CHAPTER NAME	PAGE NO
	Abstract	i
	List Of Figures	ii
	List of Output Screens	iii
	List of Tables	iii
CHAPTER 1:	**INTRODUCTION**	
1.1	Introduction	1
1.2	Existing System	3
1.3	Disadvantages of Existing System	4
1.4	Proposed System	4
1.5	Advantages Of Proposed System	4
1.6	Algorithms and flowcharts	5
CHAPTER 2:	**ANALYSIS**	
2.1	Introduction	13
2.2	Software requirements specification	13
2.2.1	Functional and Non-Functional Req	13
2.2.2	Software Requirements	15
2.2.3	Hardware Requirements	15
2.3	Data flow diagram	15
2.4	Context diagram	18
CHAPTER 3:	**DESIGN**	
3.1	Introduction	19
3.2	UML diagrams	20
3.2.1	Class Diagram	21
3.2.2	Use Case Diagram	22
3.2.3	Activity Diagram	23
3.2.4	Sequence Diagram	23

3.2.5	Collaboration Diagram	24
3.3.6	Deployment Diagram	25
3.3.7	Component Diagram	25
3.4	Module design and organization	26
CHAPTER 4:	**IMPLEMENTATION AND RESULTS**	
4.1	Introduction	27
4.2	Methods of implementation	38
CHAPTER 5:	**TESTING AND VALIDATION**	
5.1	Testing introduction	43
5.2	Design of test Cases	47
5.3	Testing and Validation	48
CHAPTER 6:	**CONCLUSION & FUTURE ENHANCEMENT**	49
CHAPTER 7:	**REFERENCES**	51

ABSTRACT

Characterizing the work that people do on their jobs is a longstanding and core issue in labour economics. Traditionally, classification has been done manually. If it were possible to combine new computational tools and administrative wage records to generate an automated crosswalk between job titles and occupations, millions of dollars could be saved in labour costs, data processing could be sped up, data could become more consistent, and it might be possible to generate, without a lag, current information about the changing occupational composition of the labour market. This application examines the potential to assign occupations to job titles contained in administrative data using automated, machine-learning approaches. We use a new extraordinarily rich and detailed set of data on transactional HR records of large firms (universities) in a relatively narrowly defined industry (public institutions of higher education) to identify the potential for machine-learning approaches to classify occupations. For this classification,n we use Naïve Bayes, Random Forest, Decision Tree, K nearest neighbour ansupportrt vector machine learning algorithms.

LIST OF FIGURES

S.NO	FIGURE NAMES	PAGE.NO
1	Context level DFD	15
2	Level 0 DFD	15
3	Level 1 DFD	16
4	Class diagram	20
5	Use Case diagram	21
6	Activity diagram	22
7	Sequence diagram	22
8	Collaboration diagram	23
9	Deployment diagram	24
10	Component diagram	24

LIST OF SCREENS

S.NO	FIGURE NAME	PAGE.NO
1	Home screen	37
2	Algorithm Selection Page	38
3	Finding Accuracy of SVM	38
4	Finding Accuracy of Random Forest	39
5	Finding Accuracy of Decision Tree	39
6	Finding Accuracy of KNN	40
7	Selecting Job Family Page	40
8	Output Prediction Page	41

LIST OF TABLES

S.NO	TABLE NAME	PAGE.NO
1.	Testing and validation	47

CHAPTER 1

INTRODUCTION

JOB CLASSIFICATION USING MACHINE LEARNING ALGORITHMS

1.1 INTRODUCTION:

Characterizing the work that people do on their jobs is a long-standing and core issue in labor economics. Traditionally, classification has been done manually, but there is a long literature on the associated challenges, well summarized by an influential paper by Mellow and Sider [(1)]. Many organizations, including the Census Bureau and the Bureau of Labor Statistics, are beginning to investigate the potential of using new computational tools, such as text analysis and machine learning, to automatically classify workers' occupations [(2)]. At the same time, there has been a surge of interest in using administrative wage records to directly capture occupations to inform training [(3)], to permit deeper longitudinal analysis on career outcomes, the effects of training, and changes in inequality. The problem is that standardized occupations do not exist on wage records, since they are drawn from human resource files which include firm specific job titles. If it were possible to combine new computational tools and administrative wage records to generate an automated crosswalk between job titles and occupations, millions of dollars could be saved in labor costs, data processing could be sped up, data could become more consistent, and it might be possible to generate, without lag, current information about the changing occupational composition of the labor market. This project examines the potential to assign occupations to job titles contained in administrative data using automated, machine-learning approaches. Although there has been little research that directly ties firm-level human resource (HR) data on job titles to occupational classifications, there are intellectual foundations for occupational coding that are largely grounded in the survey world. The first foundation is conceptual: to define each occupation. The second is operational: to translate concepts to standardized protocols. The third is statistical: to infer occupations from the information at hand. The fourth pertains to resources: the implementation of such classifications at massive scale given the limited resources available. More generally, we contribute to a much larger set

of classification problems, which are increasing in salience with the availability of more transaction data. It is important to understand which tools and approaches enable the three new rich, but unstructured data to be used, while minimizing the need for expensive and slow manual classification. We use a new extraordinarily rich and detailed set of data on transactional HR records of large firms (universities) in a relatively narrowly defined industry (public institutions of higher education) to identify the potential for machine-learning approaches to classify occupations. This is, to our knowledge, the first large-scale dataset that draws from such HR records across multiple institutions. These data have several advantages. First, the institutions are relatively large and complex, and they use HR systems that are similar to other large and complex organizations in the rest of the economy. Second, the focus on one industry limits the number of possible occupational categories, permitting a targeted analysis. Third, the focus on public universities is attractive because the HR descriptions associated with job titles are available online, and can be used to provide additional information for classification purposes. Finally, the industry is interesting in its own right. Indeed, the production of research often involves the use of intangible assets, particularly, labor inputs. Moreover, the training that students and postdocs receive is economically valuable (Zola's et al. 2015). We build a training dataset from the HR records using human duration and additional rich data sources. First, university staff and trained students manually assign occupations to job titles. That manual duration is then enhanced with additional information from online job descriptions as well as Census Bureau micro-level information on demographic characteristics and earnings. The data are then used to train machine learning models to predict occupations from job titles. Finally, the results are evaluated. While our results suggest that occupations can be assigned from job titles, they also point to real challenges. In particular, our analysis suggests that there are substantial limits to using machine learning to create discrete occupational categories, even with rich data sources. There are two core problems. The first is that occupational classifications are inherently noisy, so it is difficult to identify ground truth, particularly in a dynamic and changing economy. The second is that job titles have insufficient consistency or detail across institutions necessary for robust supervised machine learning. We do find that a large number of relatively sparsely populated job titles – a quarter of titles have only one employee and over half have fewer than ten employees

– could be assigned algorithmically, greatly reducing cost with little impact on accuracy. However, our experience suggests that, rather than trying to generate occupational categories, it might be more sensible to directly generate information about the tasks performed on jobs and the skills and experience required by each job, especially given the increased emphasis on tasks in the literature in labor economics [(5)]. The rest of the project is organized as follows. Section II provides background. Describes data and our framework. Section IV and V provide a detailed description of the first two principles, conceptual and operational, followed by statistical and implementation described in Section VI. Section VII concludes. II. Background One reason for developing occupational classifications is deeply rooted in sociology, as intrinsic to the measurement of the sources of inequality, social stratification and class mobility. Occupational classification is also essential in economic analyses, describing structural changes caused by technological advancement, automation, globalization, and change in immigration laws [(7)]. Another reason for developing occupational classification is to provide an easy-to-measure pathway from generally understood job activities to skill needs in the economy [(8)]. In practice, occupational categories were developed at scale in the 1930's and codified in the Dictionary of Occupational Titles [(9)]. Occupational analysts would "interview and observe workers, and then write job descriptions and make ratings of the characteristics of the occupation. To illustrate the magnitude of these efforts, over 75,000 on-site job analyses were conducted by analysts in the US Employment Services field centers throughout the country between the mid 1960's and 1970's alone" (Peterson et al. 2001, Since then, the Department of Labor developed NET, which provides detailed occupational taxonomies and a detailed occupation-to-skills crosswalk.

1.2 EXISTING SYSTEM:

In the existing system different machine learning algorithms are used to classify the job class commonly are Logistic regression and Multilinear regression, but these algorithms are giving less accuracy and poor results. To overcome the difficulties obtained with the above algorithms are can be done using the Naïve Bayes, Random Forest, Decision Tree, K nearest neighbour and support vector machine learning algorithm

1.3 DISADVANTAGES OF THE EXISTING SYSTEM:

- Less feature compatibility
- Low accuracy
- Poor Results
-

1.4 PROPOSED SYSTEM:

In proposed system we are using Naïve Bayes, Random Forest, Decision Tree, K nearest neighbour and Support vector machine learning algorithms. By using these algorithms we can get better accuracy with Naïve Bayes. This will give the better result and accuracy.

1.5 ADVANTAGES OF PROPOSED SYSTEM:

- Accurate classification
- Less complexity
- High performance
- Better results

1.6 ALGORITHMS AND FLOWCHARTS:

1. **Naive Bayes algorithm**

Bayes' Theorem provides a way that we can calculate the probability of a piece of data belonging to a given class, given our prior knowledge. Bayes' Theorem is stated as:

- P (class data) = (P (data class) * P(class)) / P(data)

Where P (class data) is the probability of class given the provided data.

Step 1: Separate by Class

This means that we will first need to separate our training data by class. A relatively straightforward operation.

We can create a dictionary object where each key is the class value and then add a list of all the records as the value in the dictionary.

Step 2: Summarize Dataset

We need two statistics from a given set of data.

We'll see how these statistics are used in the calculation of probabilities in a few steps. The two statistics we require from a given dataset are the mean and the standard deviation (average deviation from the mean).

The mean is the average value and can be calculated as:

- mean = sum(x)/n * count(x)

Where *x* is the list of values or a column we are looking.

Step 3: Summarize Data by Class

We require statistics from our training dataset organized by class.

Above, we have developed the *separate_by_class ()* function to separate a dataset into rows by class. And we have developed *summarize dataset ()* function to calculate summary statistics for each column.

We can put all of this together and summarize the columns in the dataset organized by class values.

Step 4: Gaussian Probability Density Function

Calculating the probability or likelihood of observing a given real-value like X1 is difficult.

One way we can do this is to assume that X1 values are drawn from a distribution, such as a bell curve or Gaussian distribution.

A Gaussian distribution can be summarized using only two numbers: the mean and the standard deviation. Therefore, with a little math, we can estimate the probability of a given value. This piece of math is called a Gaussian Probability Distribution Function (or Gaussian PDF) and can be calculated as:

- f(x) = (1 / sqrt (2 * PI) * sigma) * exp (-((x-mean) ^2 / (2 * sigma^2)))

Where *sigma* is the standard deviation for *x*, *mean* is the mean for *x* and *PI* is the value of pi.

2. Support vector classifiers algorithm

Support Vector Machine or SVM algorithm is a simple yet powerful Supervised Machine Learning algorithm that can be used for building both regression and classification models. SVM algorithm can perform really well with both linearly separable and non-linearly separable datasets. Even with a limited amount of data, the support vector machine algorithm does not fail to show its magic.

Step 1: Load Pandas library and the dataset using Pandas

Step 2: Define the features and the target

Step 3: Split the dataset into train and test using sklearn before building the SVM algorithm model

Step 4: Import the support vector classifier function or SVC function from Sklearn SVM module. Build the Support Vector Machine model with the help of the SVC function

Step 5: Predict values using the SVM algorithm model

Step 6: Evaluate the Support Vector Machine model

3. K-NEAREST NEIGHBOUR:

This k-Nearest Neighbors tutorial is broken down into 3 parts:

Step 1: Calculate Euclidean Distance.

Step 2: Get Nearest Neighbors.

Step 3: Make Predictions

Step 1: Calculate Euclidean Distance.

He first step is to calculate the distance between two rows in a dataset.

Rows of data are mostly made up of numbers and an easy way to calculate the distance between two rows or vectors of numbers is to draw a straight line. This makes sense in 2D or 3D and scales nicely to higher dimensions

Euclidean Distance = sqrt(sum i to N (x1_i – x2_i)^2)

Where *x1* is the first row of data, *x2* is the second row of data and *i* is the index to a specific column as we sum across all columns.

With Euclidean distance, the smaller the value, the more similar two records will be. A value of 0 means that there is no difference between two records

Step 2: Get Nearest Neighbours**:**

Neighbors for a new piece of data in the dataset are the *k* closest instances, as defined by our distance measure. To locate the neighbors for a new piece of data within a dataset we must first calculate the distance between each record in the dataset to the new piece of data. We can do this using our distance function prepared above.

Once distances are calculated, we must sort all of the records in the training dataset by their distance to the new data. We can then select the top *k* to return as the most similar neighbors.

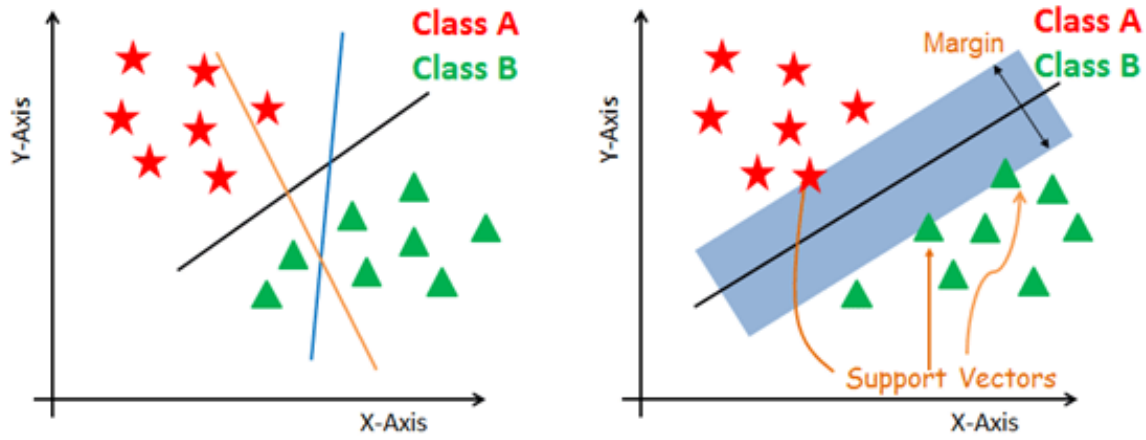
We can do this by keeping track of the distance for each record in the dataset as a tuple, sort the list of tuples by the distance (in descending order) and then retrieve the neighbor.

Step 3: Make Predictions

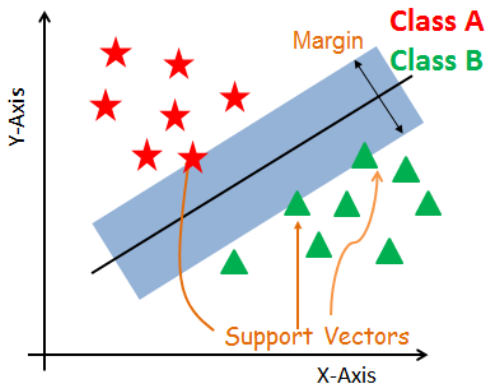
The most similar neighbors collected from the training dataset can be used to make predictions.

In the case of classification, we can return the most represented class among the neighbors. We can achieve this by performing the *max()* function on the list of output values from the neighbours. Given a list of class values observed in the neighbours, the *max()* function takes a set of unique class values and calls the count on the list of class values for each class value in the set.

4. SVM:



Generally, Support Vector Machines (SVM) is considered to be a classification approach but it can be employed in both types of classification and regression problems. It can easily handle multiple continuous and categorical variables. SVM constructs a hyper plane in multidimensional space to separate different classes. SVM generates optimal hyper plane in an iterative manner, which is used to minimize an error. The core idea of SVM is to find a maximum marginal hyper plane (MMH) that best divides the dataset into classes.



SVM is a very good algorithm for doing classification. It's a supervised learning algorithm that is mainly used to classify data into different classes. SVM trains on a set of label data. The main advantage of SVM is that it can be used for both classification and regression problems. SVM draws a decision boundary which is a hyper plane between any two classes in order to separate them or classify them. SVM also used in Object Detection and image classification.

Here, I am going to use the Cats & Dogs dataset for doing Classification using SVM. You can collect the dataset from here. It's a binary classification problem, but Support Vector Machine can also be used for multiclass classification problems.

Support Vectors

Support vectors are the data points, which are closest to the hyper plane. These points will define the separating line better by calculating margins. These points are more relevant to the construction of the classifier.

Hyper plane

A hyper plane is a decision plane that separates between a set of objects having different class memberships.

Margin

A margin is a gap between the two lines on the closest class points. This is calculated as the perpendicular distance from the line to support vectors or closest points. If the margin is larger in between the classes, then it is considered a good margin, a smaller margin is a bad margin.

How does SVM work?

The main objective is to segregate the given dataset in the best possible way. The distance between the either nearest point is known as the margin. The objective is to select a hyper plane with the maximum possible margin between support vectors in the given dataset. SVM searches for the maximum marginal hyper plane in the following steps:

1. Generate hyper planes that segregate the classes in the best way. Left-hand side figure showing three hyper planes black, blue, and orange. Here, the blue and orange have higher classification errors, but the black is separating the two classes correctly.
2. Select the right hyper plane with the maximum segregation from the either nearest data points as shown in the right-hand side figure.

5. **Random Forest:**

Ensemble methods is a machine learning technique that combines several base models in order to produce one optimal predictive model. To better understand this definition lets take a step back into ultimate goal of machine learning and model building. This is going to make more sense as I dive into specific examples and why Ensemble methods are used.

I will largely utilize Decision Trees to outline the definition and practicality of Ensemble Methods (however it is important to note that Ensemble Methods do not only pertain to Decision Trees).

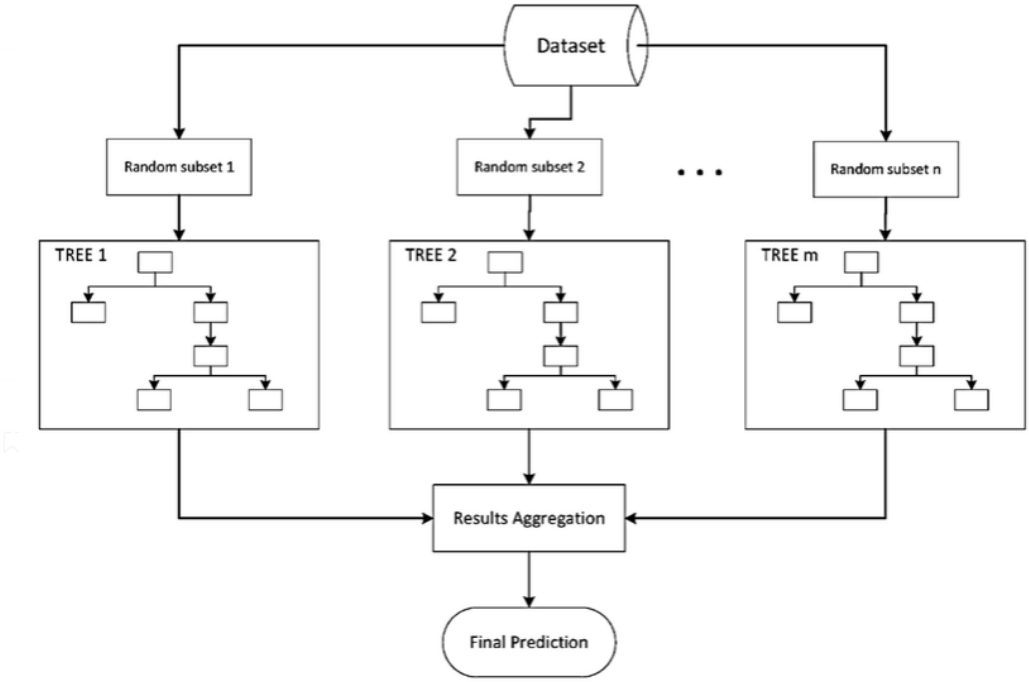
A Decision Tree determines the predictive value based on series of questions and conditions. For instance, this simple Decision Tree determining on whether an individual should play outside or not. The tree takes several weather factors into account, and given each factor either makes a decision or asks another question. In this example, every time it is overcast, we will play outside.

However, if it is raining, we must ask if it is windy or not? If windy, we will not play. But given no wind, tie those shoelaces tight because were going outside to play.

When making Decision Trees, there are several factors we must take into consideration: On what features do we make our decisions on? What is the threshold for classifying each question into a yes or no answer? In the first Decision Tree, what if we wanted to ask ourselves if we had friends to play with or not. If we have friends, we will play every time. If not, we might continue to ask ourselves questions about the weather. By adding an additional question, we hope to greater define the Yes and No classes.

This is where Ensemble Methods come in handy! Rather than just relying on one Decision Tree and hoping we made the right decision at each split, Ensemble Methods allow us to take a sample of Decision Trees into account, calculate which features to use or questions to ask at each split, and make a final predictor based on the aggregated results of the sampled Decision Trees.

*Bagg*ing, or *B*ootstrap *Agg*regating. Bagg**i**ng gets its name because it combines *B*ootstrapping and *Agg*regation to form one ensemble model. Given a sample of data, multiple bootstrapped subsamples are pulled. A Decision Tree is formed on each of the bootstrapped subsamples. After each subsample Decision Tree has been formed, an algorithm is used to aggregate over the Decision Trees to form the most efficient predictor. The image below will help explain:



Random Forest Models: Random Forest Models can be thought of as Bagging, with a slight tweak. When deciding where to split and how to make decisions, Bagged Decision Trees have the full disposal of features to choose from. Therefore, although the bootstrapped samples may be slightly different, the data is largely going to break off at the same features throughout each model. In contrary, Random Forest models decide where to split based on a random selection of features. Rather than splitting at similar features at each node throughout, Random Forest models implement a level of differentiation because each tree will split based on different features. This level of differentiation provides a greater ensemble to aggregate over, ergo producing a more accurate predictor. Refer to the image for a better understanding.

CHAPTER 2
ANALYSIS

2.1 INTRODUCTION:

In the existing system different machine learning algorithms are used to classify the job class commonly are Logistic regression and Multi linear regression, but these algorithms are giving less accuracy and poor results. To overcome the difficulties obtained with the above algorithms are can be done using the Naïve Bayes, Random Forest, Decision Tree, K nearest neighbour and Support vector machine learning algorithm.

2.2 SOFTWARE REQUIREMENT SPECIFICATION:

2.2.1 Functional and non-functional requirements:

Requirement's analysis is very critical process that enables the success of a system or software project to be assessed. Requirements are generally split into two types: Functional and non-functional requirements.

Functional Requirements: These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

Examples of functional requirements:

1) Authentication of user whenever he/she logs into the system
2) System shutdown in case of a cyber-attack
3) A verification link is sent to the user. User after clicking on the link it navigates to project frontend.



Fig: i

Non-functional requirements: These are basically the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements.

They basically deal with issues like:

- **Speed:** Speed determines how fast our web framework responds to commands.
- **Security:** To protect sensitive data, you may consider developing nonfunctional security features. For example, how our frontend framework is protected

  Account locking: After a certain number of login attempts, a security system may lock an account to protect a user's information from potential hackers.
- **Portability:** Portability means how effectively a system performs in one environment compared to another.
- **Environment:** The environment includes external factors that impact how your system performs. For example, humid weather and exposure to water may affect the speed or reliability of an application.

Examples of non-functional requirements:

1) Emails should be sent with a latency of no greater than 12 hours from such an activity.
2) The processing of each request should be done within 10 seconds
3) The site should load in 3 seconds whenever of simultaneous users are > 10000

2.2.2 SOFTWARE REQUIREMENTS:

- Operating System : Windows 10
- Server-side Script : Python 3.6
- IDE : PyCharm
- Framework : Stream Lit
- Libraries Used : NumPy, Pandas, Scikit-Learn.

2.2.3 HARDWARE REQUIREMENTS:

- Processor : I3/Intel Processor
- RAM : 8GB
- Hard Disk : 128 GB

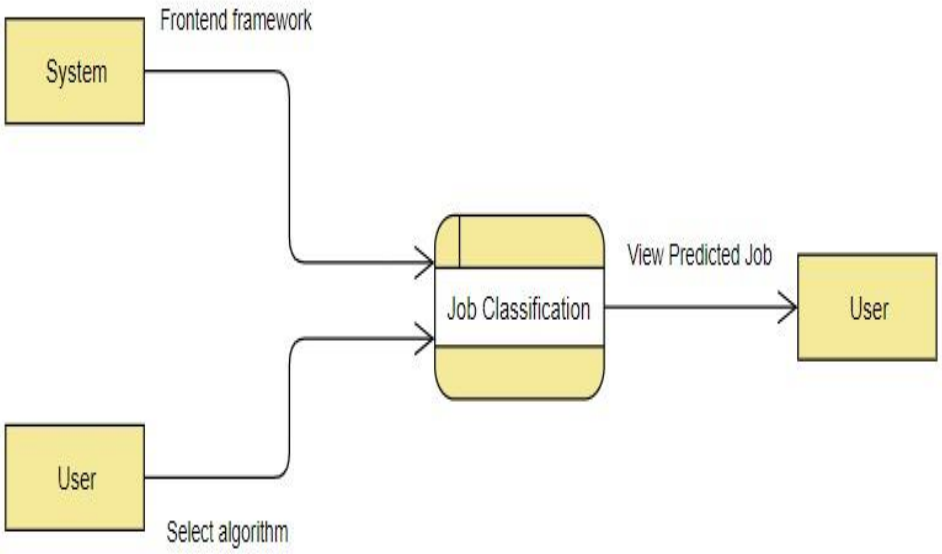
2.3 DATA FLOW DIAGRAM:

A Data Flow Diagram (DFD) is a traditional way to visualize the information flows within a system. A neat and clear DFD can depict a good amount of the system requirements graphically. It can be manual, automated, or a combination of both. It shows how information enters and leaves the system, what changes the information and where information is stored. The purpose of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communications tool between a systems analyst and any person who plays a part in the system that acts as the starting point for redesigning a system.

Data Flow Diagram symbols are standardized notations, like rectangles, circles, arrows, and short-text labels, that describe a system or process' data flow direction, data inputs, data outputs, data storage points, and its various sub-processes.

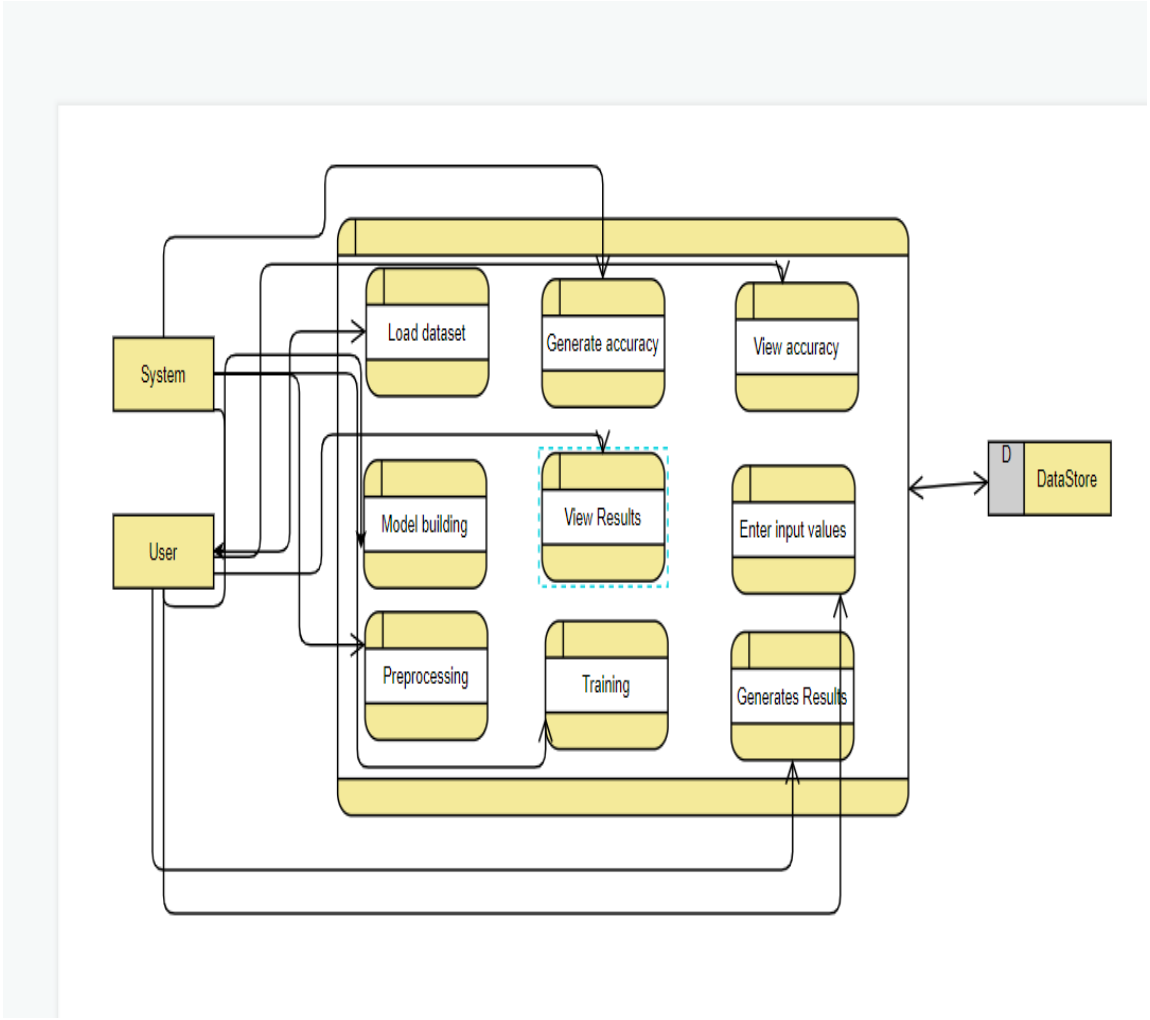
LEVEL 0 DFD:

DFD Level 0 is also called a Context Diagram. It's a basic overview of the whole system or process being analyzed or modeled. It's designed to be an at-a-glance view, showing the system as a single high-level process, with its relationship to external entities.



Level 1 DFD:

DFD Level 1 provides a more detailed breakout of pieces of the Context Level Diagram. You will highlight the main functions carried out by the system, as you break down the high-level process of the Context Diagram into its subprocesses.



2.4 CONTEXT DIAGRAM OF PROJECT:

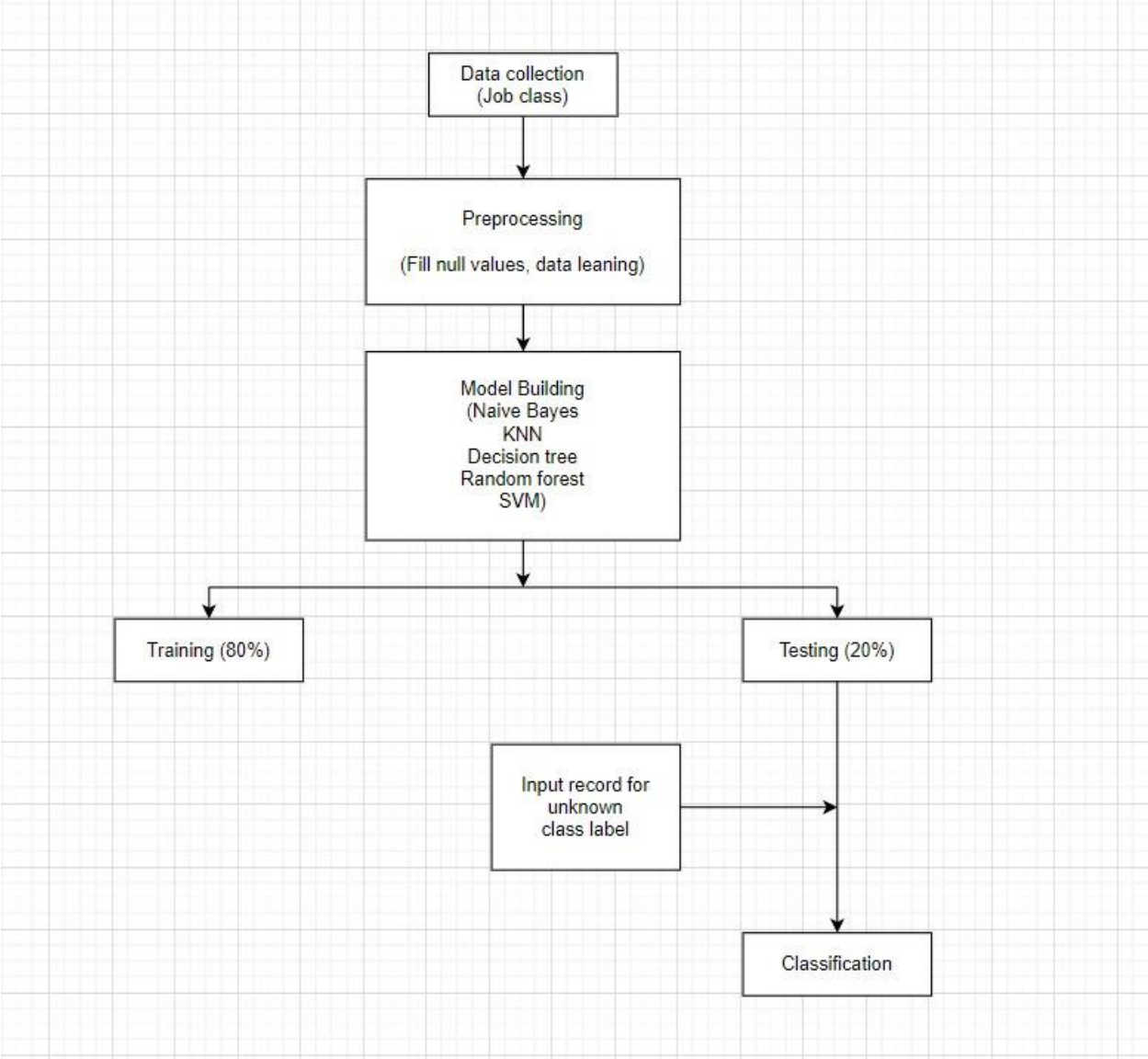


Fig: ii

CHAPTER 3

DESIGN

3.1 INTRODUCTION:

Input Design:

In an information system, input is the raw data that is processed to produce output. During the input design, the developers must consider the input devices such as PC, MICR, OMR, etc.

Therefore, the quality of system input determines the quality of system output. Well-designed input forms and screens have following properties −

- It should serve specific purpose effectively such as storing, recording, and retrieving the information.
- It ensures proper completion with accuracy.
- It should be easy to fill and straightforward.
- It should focus on user's attention, consistency, and simplicity.
- All these objectives are obtained using the knowledge of basic design principles regarding −
    - What are the inputs needed for the system?
    - How end users respond to different elements of forms and screens.

Objectives for Input Design:

The objectives of input design are

- To design data entry and input procedures
- To reduce input volume
- To design source documents for data capture or devise other data capture methods

- To design input data records, data entry screens, user interface screens, etc.
- To use validation checks and develop effective input controls.

Output Design:

The design of output is the most important task of any system. During output design, developers identify the type of outputs needed, and consider the necessary output controls and prototype report layouts.

Objectives of Output Design:

The objectives of input design are:

- To develop output design that serves the intended purpose and eliminates the production of unwanted output.
- To develop the output design that meets the end user's requirements.
- To deliver the appropriate quantity of output.
- To form the output in appropriate format and direct it to the right person.
- To make the output available on time for making good decisions.

3.2 UML DIAGRAMS:

UML DIAGRAMS:

UML stands for Unified Modelling Language. UML is a standardized general-purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modelling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modelling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

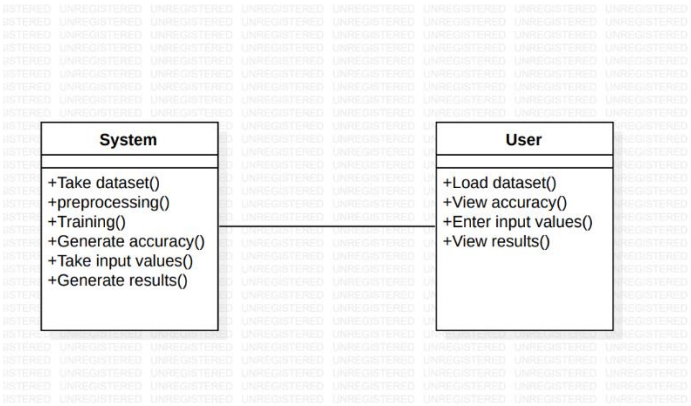
GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modelling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

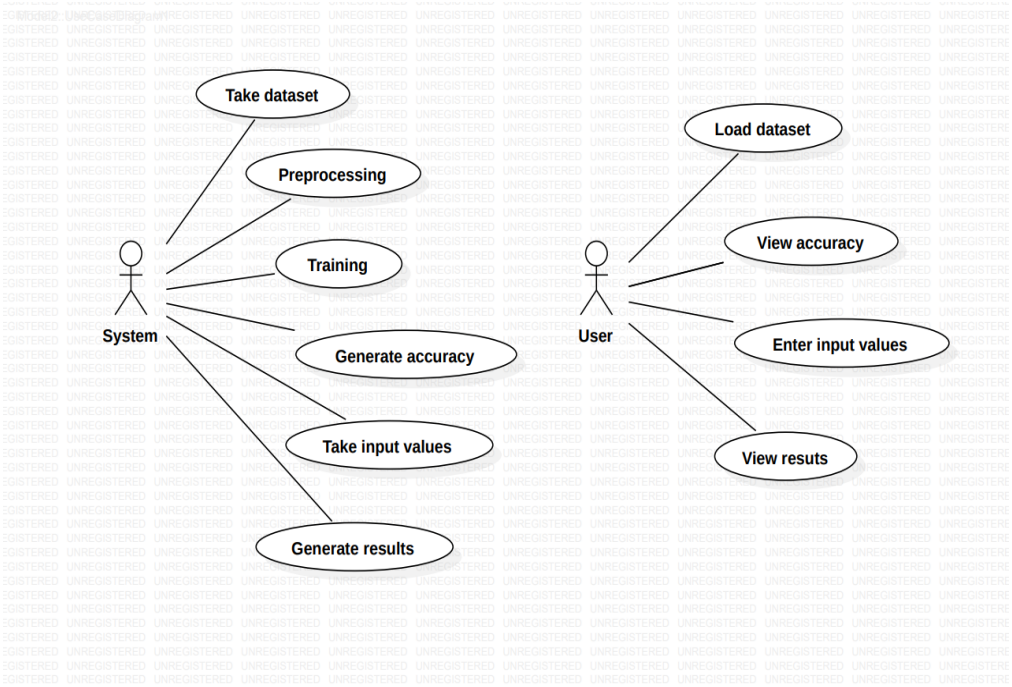
3.2.1 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information



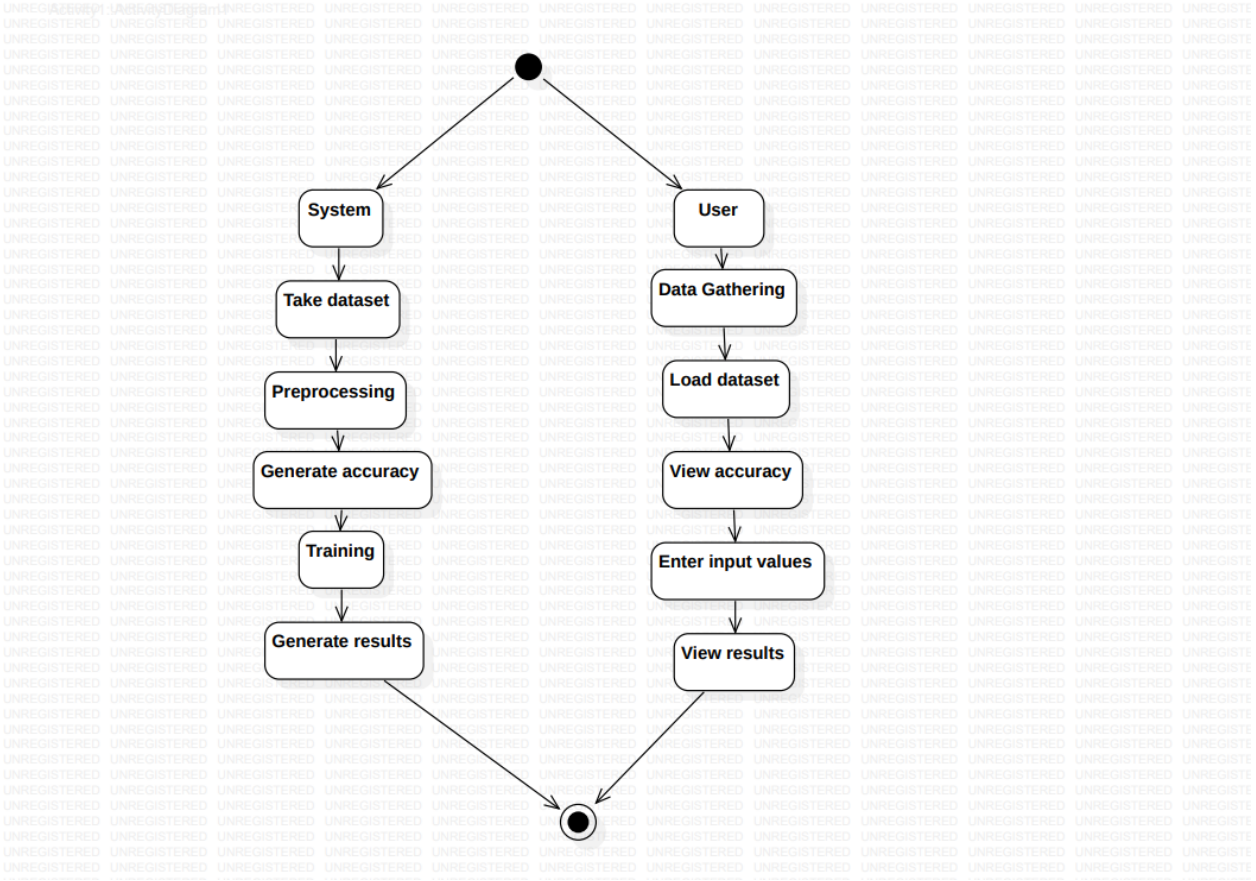
3.2.2 USE CASE DIAGRAM

- A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis.
- Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases.
- The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



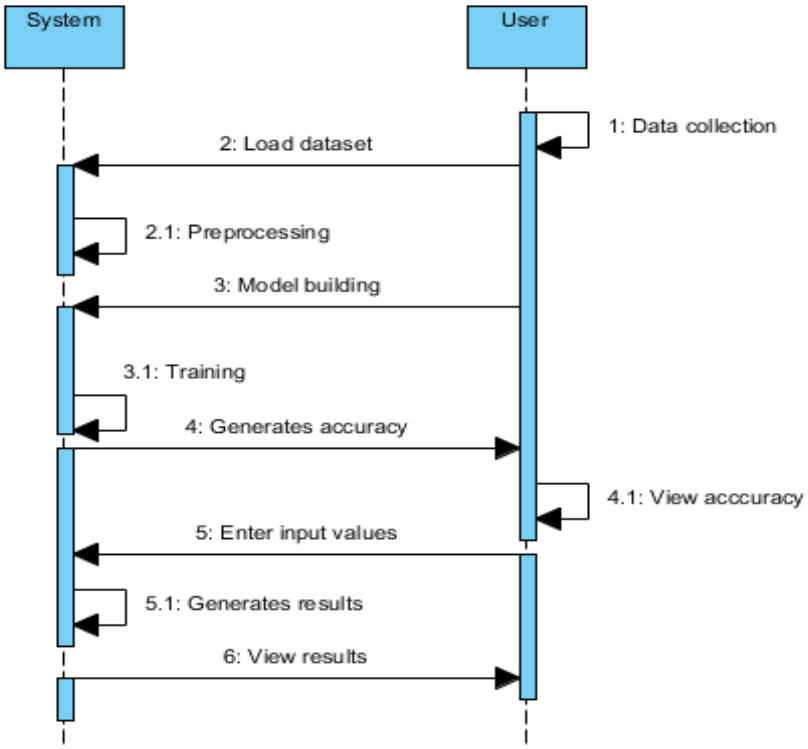
3.2.3 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



3.2.4 SEQUENCE DIAGRAM

- A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order.
- It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams



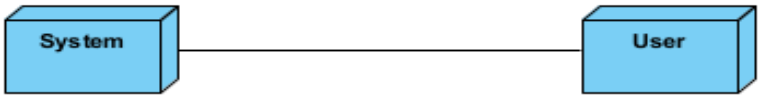
3.2.5 COLLABORATION DIAGRAM:

In collaboration diagram the method call sequence is indicated by some numbering technique as shown below. The number indicates how the methods are called one after another. We have taken the same order management system to describe the collaboration diagram. The method calls are similar to that of a sequence diagram. But the difference is that the sequence diagram does not describe the object organization whereas the collaboration diagram shows the object organization.



3.2.6 DEPLOYMENT DIAGRAM

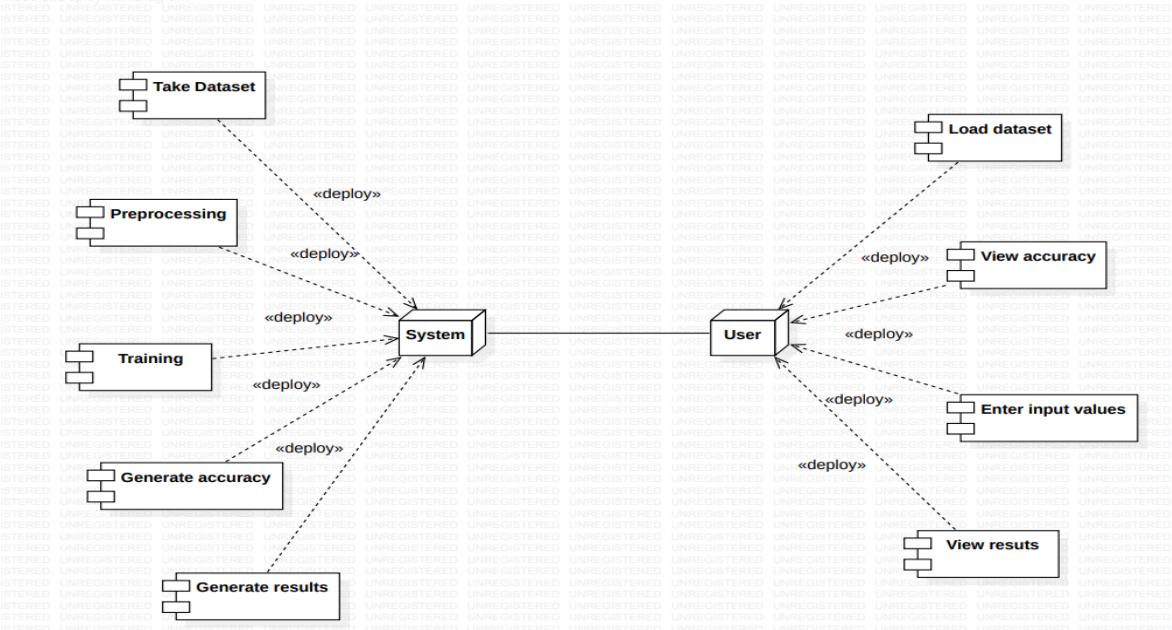
Deployment diagram represents the deployment view of a system. It is related to the component diagram. Because the components are deployed using the deployment diagrams. A deployment diagram consists of nodes. Nodes are nothing but physical hardware's used to deploy the application.



3.2.7 COMPONENT DIAGRAM:

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical **c**omponents in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required function is covered by planned development.

z



3.4 MODULE DESIGN AND ORGANIZATION:

1. System:

1.1 Pre-processing:

In this step data cleaning and data filling is done.

1.2 Training:

Use the pre-processed training dataset is used to train our machine learning algorithms.

1.3 Generate accuracy

System generates accuracy for our model and dataset. This tells us how much efficiently model is working.

1.3 Generates results:

The results will be displayed after prediction.

2. User:

2.1 Data collection

The user has to upload an dataset which needs to be classified.

2.2 Model building

User builds the models to fit our data for prediction of job class.

2.3 View Accuracy

Users view the generated accuracy from the system.

2.4 View Results

Users can view the generated classification from the user.

CHAPTER 4
IMPLEMENTATION & RESULTS

4.1 INTRODUCTION

Implementation is the stage in the project where the theoretical design is turned into the working system and is giving confidence to the new system for the users i.e. will work efficiently and effectively. It involves careful planning, investigation of the current system and its constraints on implementation, design of method to achieve the change over, an evaluation, of change over methods. Apart from planning, a major task of preparing the implementation is education of users. The more complex system is implemented, the more involved will be the system analysis and design effort required just for implementation. An implementation coordinating committee based on policies of individual organizations has been appointed. The implementation process begins with preparing a plan for the implementation for the system. According to this plan, the activities are to be carried out; discussions may regarding the equipment have to be acquired to implement the new system.

Implementation is the final and important phase. The most critical stage is in achieving a successful new system and in giving the users confidence that the new system will work and be effective. The system can be implemented only after thorough testing is done and if it is found to work according to the specification. This method also offers the greatest security since the old system can take over if the errors are found or inability to handle certain types of transaction while using the new system.

At the beginning of the development phase a preliminary implementation plan is created to schedule and manage the many different activities that must be integrated into the plan. The implementation plan is updated throughout the Development phase, culminating in a changeover plan for the operation phase. The major elements of the implementation plan are the test plan, training plan, equipment installation plan, and a conversion plan.

ABOUT PYTHON

What is Python?

Chances you are asking yourself this. You may have found this book because you want to learn to program but don't know anything about programming languages. Or you may have heard of programming languages like C, C++, C#, or Java and want to know what Python is and how it compares to "big name" languages. Hopefully I can explain it for you.

Python concepts:

If you're not interested in the how and whys of Python, feel free to skip to the next chapter. In this chapter I will try to explain to the reader why I think Python is one of the best languages available and why it's a great one to start programming with.

- Open-source general-purpose language.
- Object Oriented, Procedural, Functional.
- Easy to interface with C/ObjC/Java/Fortran.
- Easy-is to interface with C++ (via SWIG).
- Great interactive environment

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

- **Python is Object-Oriented** − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Python Features:

Python's features include

- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.
- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − you can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** − Python provides interfaces to all major commercial databases.
- **GUI Programming** − Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** − Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below:

- It supports functional and structured programming methods as well as OOP.

- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.

NumPy

Humpy's main object is the homogeneous multidimensional array. It is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers. In numbly dimensions are called axes. The number of axes is rank.

- Offers Matlab-ish capabilities within Python.
- Fast array operations.
- 2D arrays, multi-D arrays, linear algebra etc.

Matplotlib

- High quality plotting library.

Software Installation:

Installing Python:

1. To download and install Python visit the official website of Python https://www.python.org/downloads/ and choose your version.

2. Once the download is complete, run the exe for install Python. Now click on Install Now.

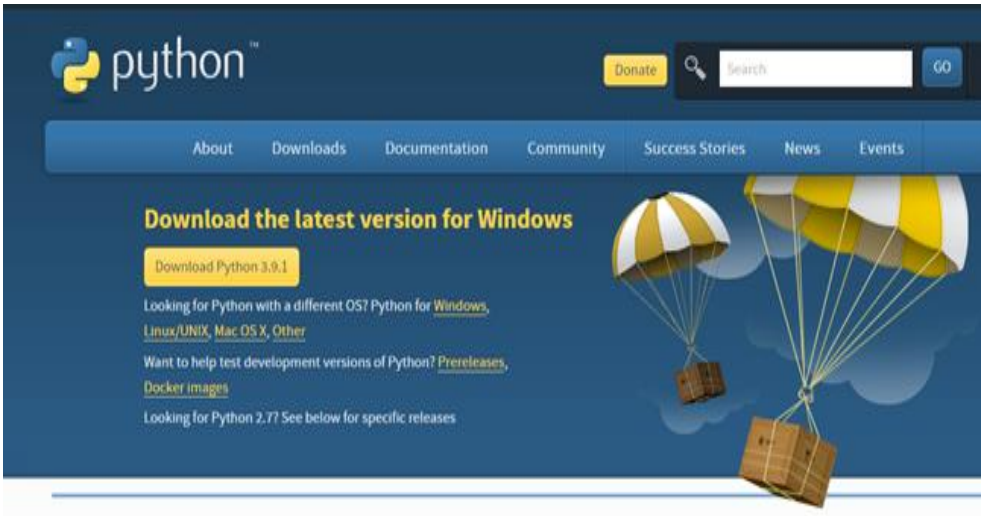


Fig: Python download website

3. You can see Python installing at this point.

4. When it finishes, you can see a screen that says the Setup was successful. Now click on "Close".

Installing PyCharm:

1.To download PyCharm visit the website https://www.jetbrains.com/pycharm/download/ and click the "DOWNLOAD" link under the Community Section.



Fig: Downloading PyCharm

1. Once the download is complete, run the exe for install PyCharm. The setup wizard should have started. Click “Next”.
2. On the next screen, Change the installation path if required. Click “Next”.
3. On the next screen, you can create a desktop shortcut if you want and click on “Next”.
4. Choose the start menu folder. Keep selected Jet Brains and click on “Install”.
5. Wait for the installation to finish.
6. Once installation finished, you should receive a message screen that PyCharm is installed. If you want to go ahead and run it, click the “Run PyCharm Community Edition” box first and click “Finish”.
7. After you click on "Finish," the Following screen will appear.

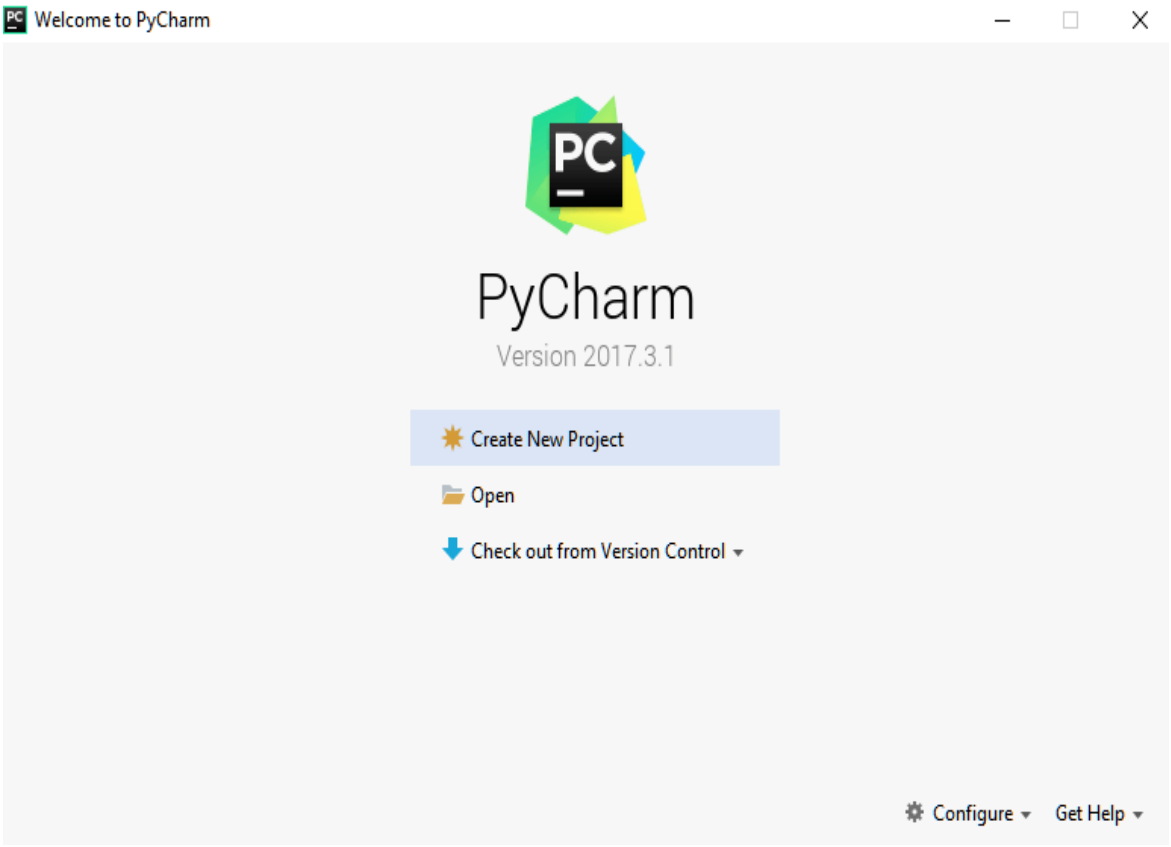


Fig: PyCharm home screen

9. You need to install some packages to execute your project in a proper way.

10. Open the command prompt/ anaconda prompt or terminal as administrator.

11. The prompt will get open, with specified path, type “pip install package name” which you want to install (like Numpy, pandas, seaborn, scikit-learn, matplotlib.pyplot)

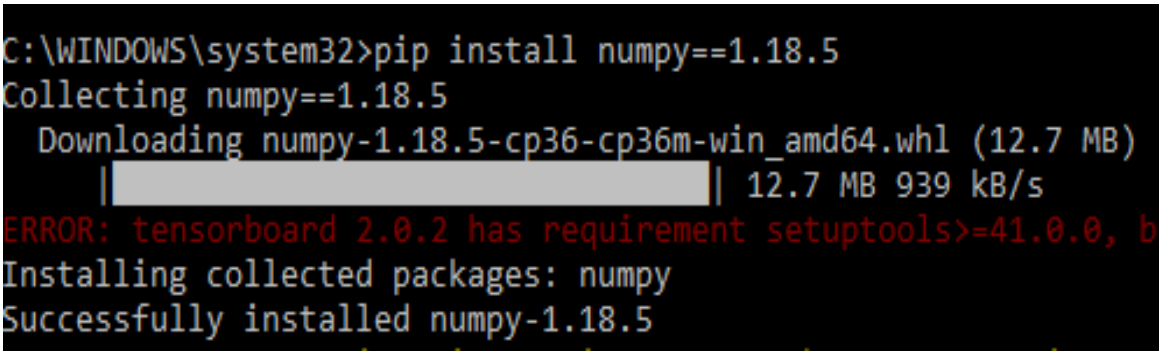


Fig: Installing NumPy

Python modules:

Python allows us to store our code in files (also called modules). This is very useful for more serious programming, where we do not want to retype a long function definition from the very beginning just to change one mistake. In doing this, we are essentially defining our own modules, just like the modules defined already in the Python library.

To support this, Python has a way to put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a module; definitions from a module can be imported into other modules or into the main module.

Main.py: This file contains main code of project.

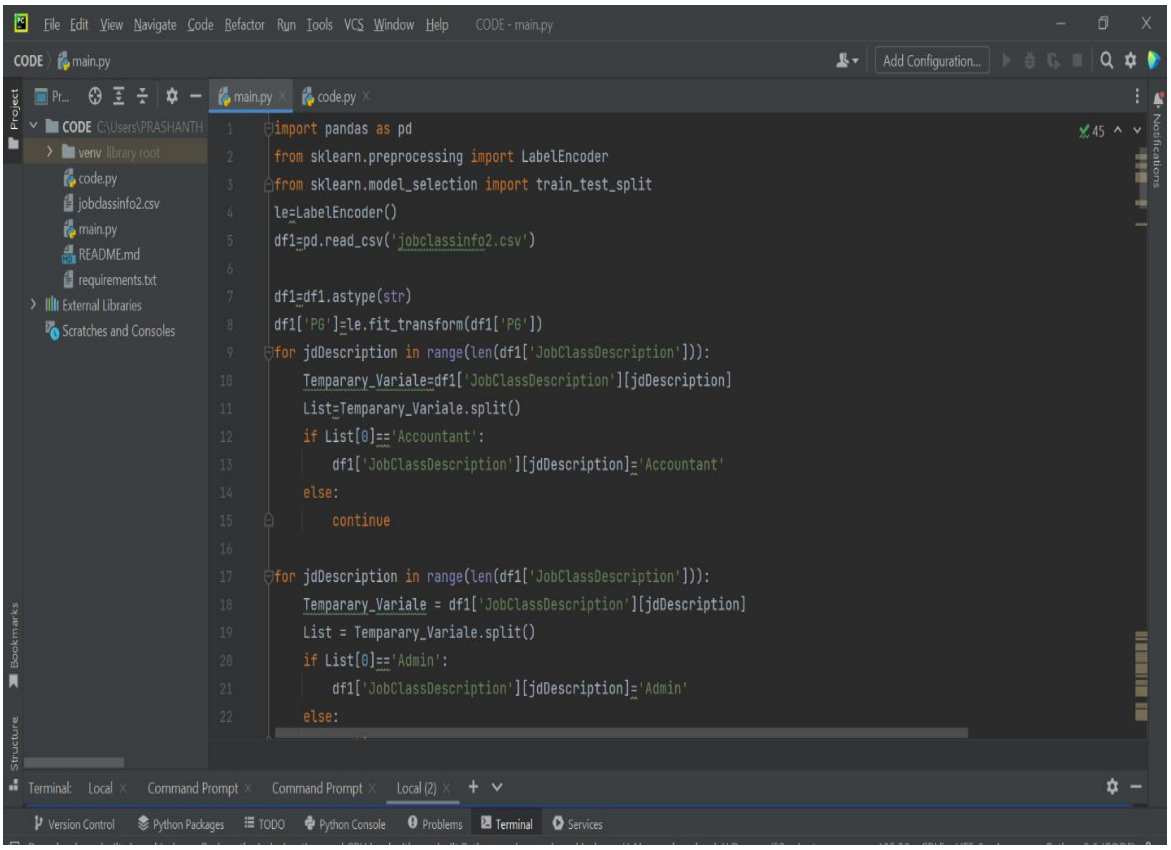

File Edit View Navigate Code Refactor Run Tools VCS Window Help
CODE - main.py
CODE main.py
Add Configuration...
main.py
code.py
CODE C:\Users\PRASHANTH
venv library root
code.py
jobclassinfo2.csv
main.py
README.md
requirements.txt
External Libraries
Scratches and Consoles
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
le=LabelEncoder()
df1=pd.read_csv('jobclassinfo2.csv')
df1=df1.astype(str)
df1['PG']=le.fit_transform(df1['PG'])
for jdDescription in range(len(df1['JobClassDescription'])):
Temparary_Variale=df1['JobClassDescription'][jdDescription]
List=Temparary_Variale.split()
if List[0]=='Accountant':
df1['JobClassDescription'][jdDescription]='Accountant'
else:
continue
for jdDescription in range(len(df1['JobClassDescription'])):
Temparary_Variale = df1['JobClassDescription'][jdDescription]
List = Temparary_Variale.split()
if List[0]=='Admin':
df1['JobClassDescription'][jdDescription]='Admin'
else:
Terminal: Local Command Prompt Command Prompt Local (2)
Version Control Python Packages TODO Python Console Problems Terminal Services

Fig: i

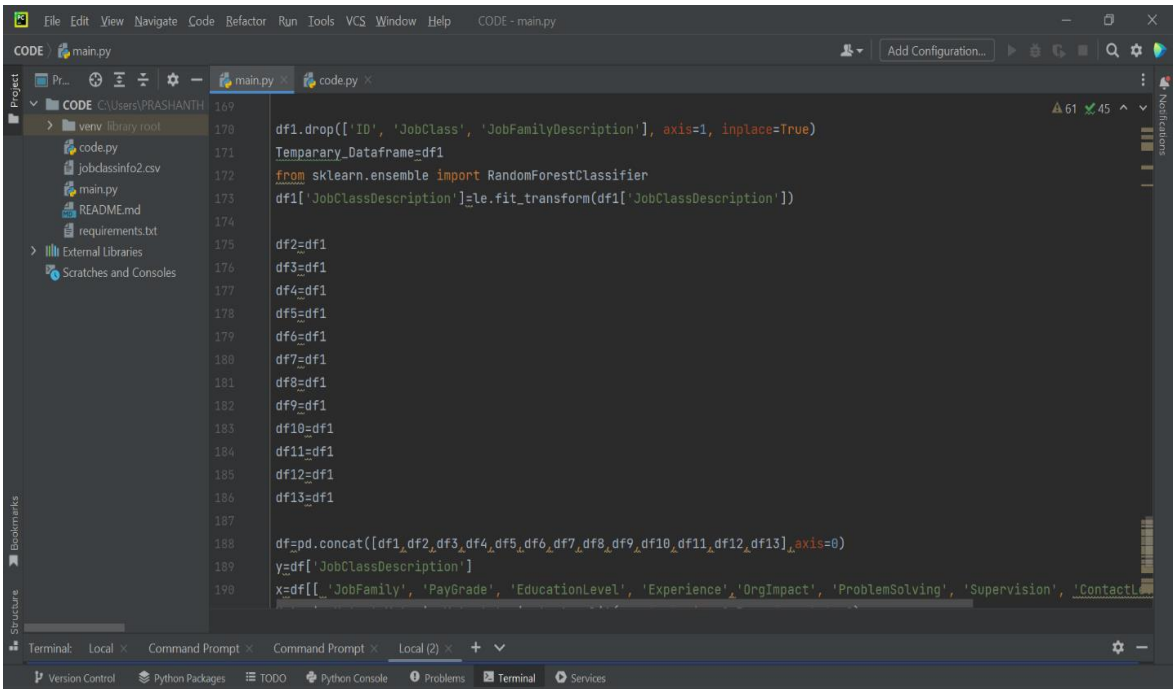

File Edit View Navigate Code Refactor Run Tools VCS Window Help
CODE - main.py
CODE main.py
Add Configuration...
main.py
code.py
df1.drop(['ID', 'JobClass', 'JobFamilyDescription'], axis=1, inplace=True)
Temparary_Dataframe=df1
from sklearn.ensemble import RandomForestClassifier
df1['JobClassDescription']=le.fit_transform(df1['JobClassDescription'])
df2=df1
df3=df1
df4=df1
df5=df1
df6=df1
df7=df1
df8=df1
df9=df1
df10=df1
df11=df1
df12=df1
df13=df1
df=pd.concat([df1,df2,df3,df4,df5,df6,df7,df8,df9,df10,df11,df12,df13],axis=0)
y=df['JobClassDescription']
x=df[['JobFamily', 'PayGrade', 'EducationLevel', 'Experience','OrgImpact', 'ProblemSolving', 'Supervision', 'ContactL
Terminal: Local Command Prompt Command Prompt Local (2)
Version Control Python Packages TODO Python Console Problems Terminal Services

Fig: ii

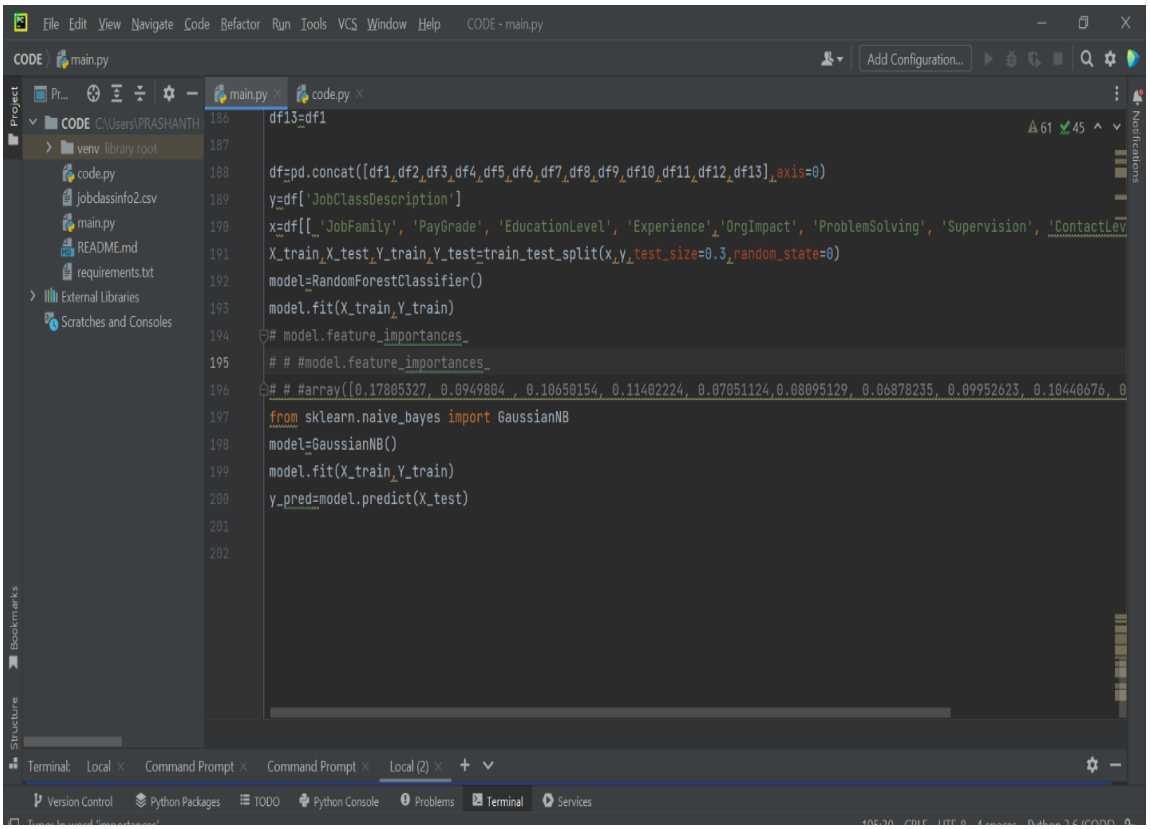


Fig: iii

Functions in Python:

It is possible, and very useful, to define our own functions in Python. Generally speaking, if you need to do a calculation only once, then use the interpreter. But when you or others have need to perform a certain type of calculation many times, then define a function.

You use functions in programming to bundle a set of instructions that you want to use repeatedly or that, because of their complexity, are better self-contained in a sub-program and called when needed. That means that a function is a piece of code written to carry out a specified task. To carry out that specific task, the function might or might not need multiple inputs. When the task is carved out, the function can or cannot return one or more values.

There are three types of functions in python: Help (), min (), print ().

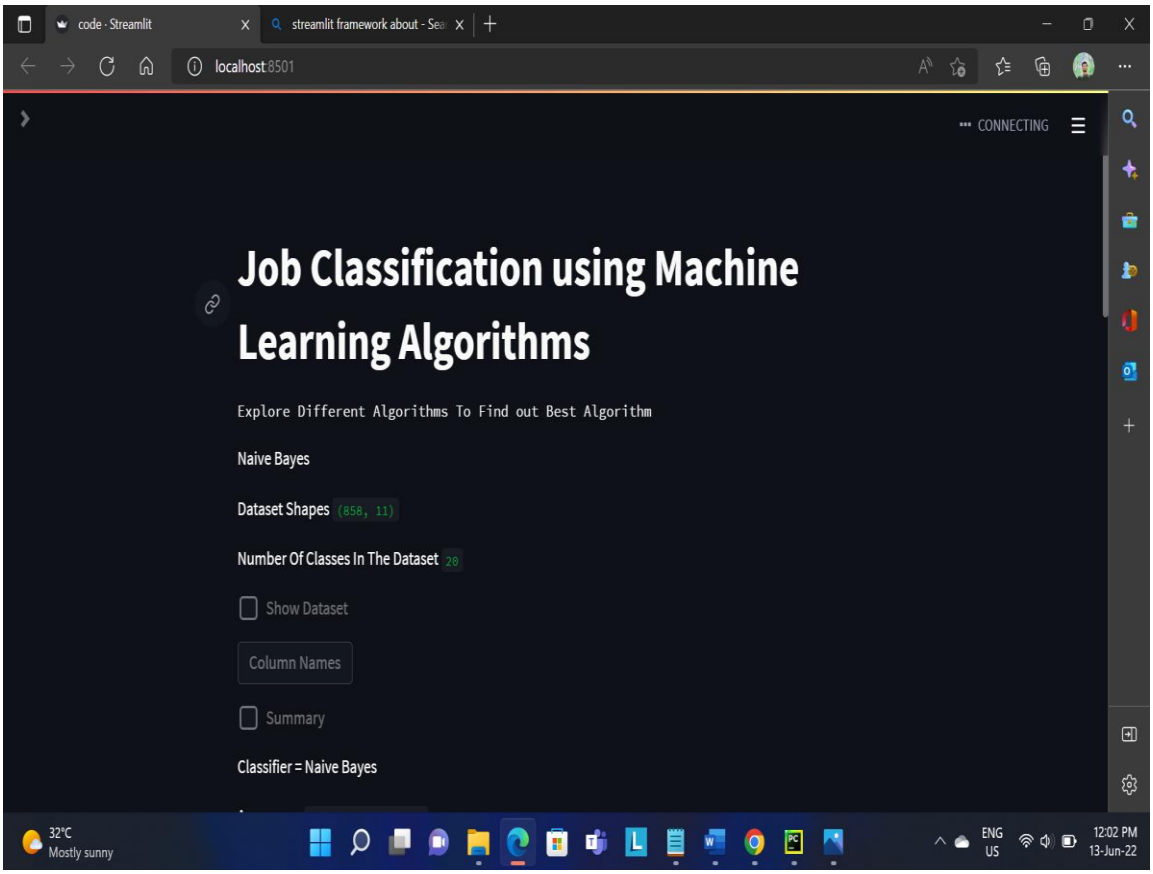
Python Web Frameworks:

A web framework is a code library that makes a developer's life easier when building reliable, scalable and maintainable web applications.

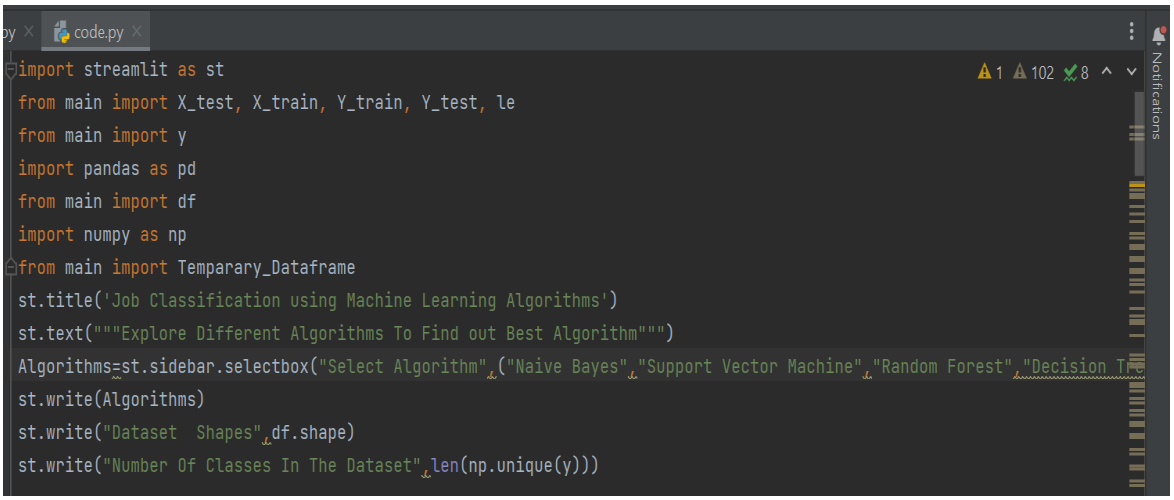
Web Framework:

Streamlit is a web application framework, that helps us build and develop Python-based web applications that can be used to share analytics results, build complex interactive experiences, and illustrate new machine learning models.

SCREEN 1: Home page of our web framework



SCREEN 2: Importing web Framework



Why are web frameworks useful?

Web frameworks encapsulate what developers have learned over the past twenty years while programming sites and applications for the web. Frameworks make it easier to reuse code for common HTTP operations and to structure projects so other developers with knowledge of the framework can quickly build and maintain the application.
Frameworks provide functionality in their code or through extensions to perform common operations required to run web applications. These common operations include:

- URL routing.
- HTML, XML, JSON, and other output format tinplating.
- Database manipulation.
- Security against Cross-site request forgery (CSRF) and other attacks.

Not all web frameworks include code for all of the above functionality. Frameworks fall on the spectrum from executing a single use case to providing every known web framework feature to every developer. Some frameworks take the "batteries-included" approach where everything possible comes bundled with the framework while others have a minimal core package that is amenable to extensions provided by other packages.

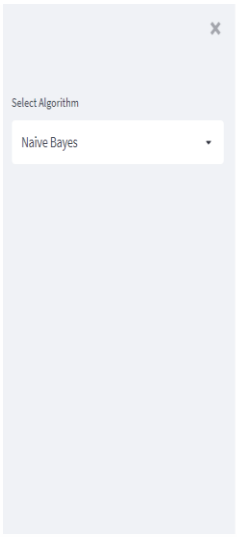
Steps for Executing the Project

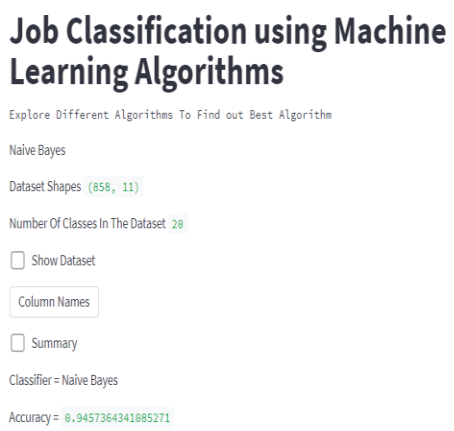
1. Install the required packages.
2. Create a registration page for job classification.
3. Click on view data set.
4. Select required algorithm to check Accuracy.
5. Click on view accuracy.
6. Select type of job required to predict from job family description.
7. Enter input values in the home page.
8. View the results.
9. Logout from admin page.

4.2 METHODS OF IMPLEMENTATION:

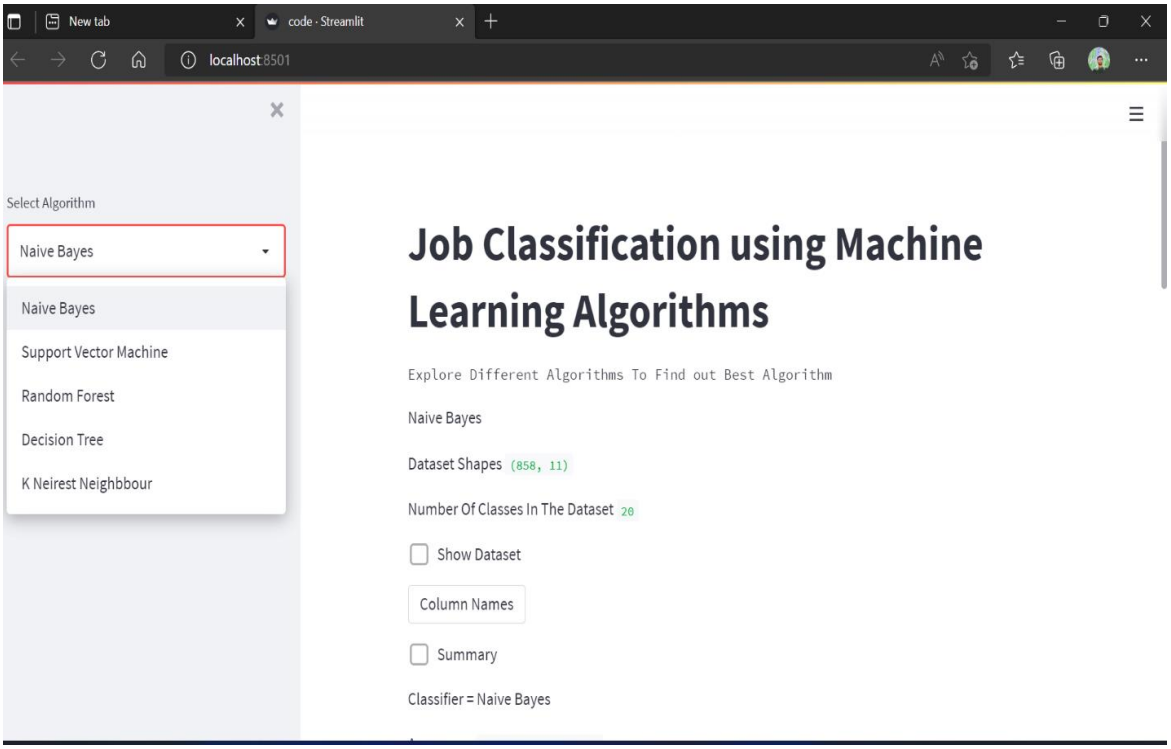
SCREEN 1: Home Page

The accuracy of the naïve Bayes algorithm is achieving 94.3%.



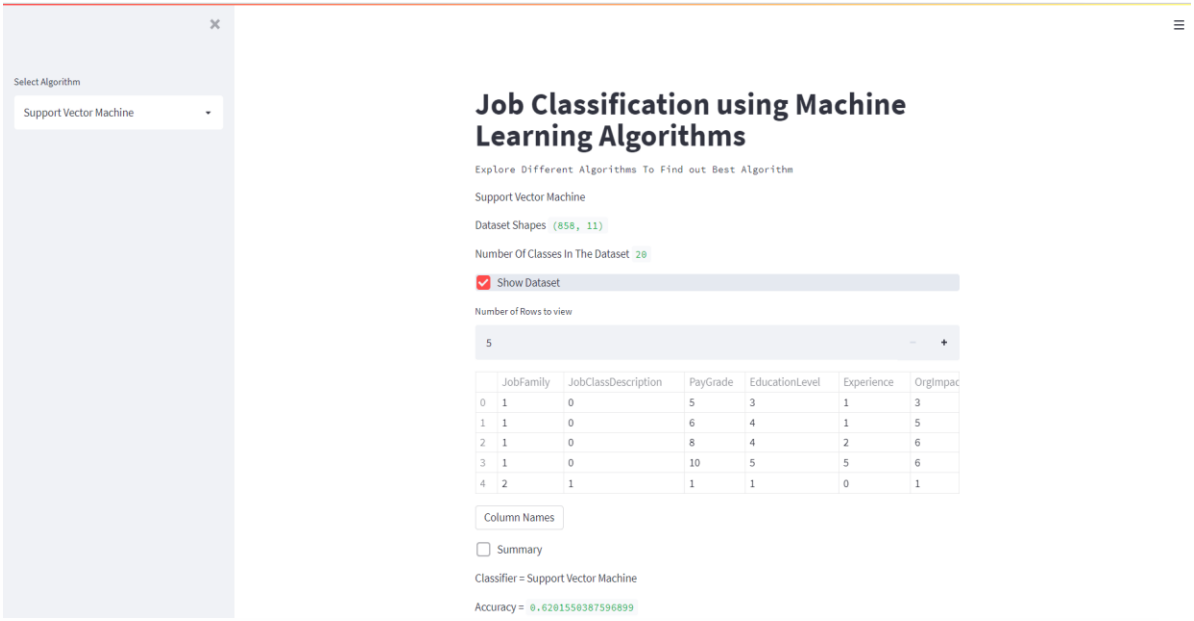


SCREEN 2: Selecting algorithm



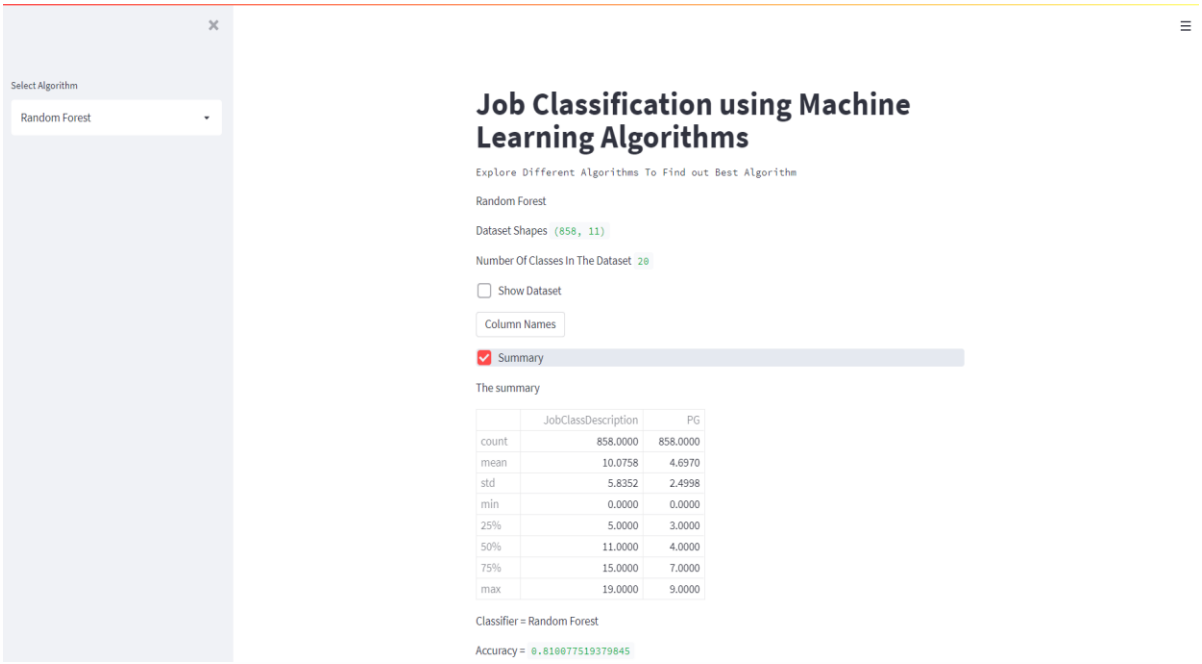
SCREEN 3: Accuracy of SVM

The accuracy of the SVM is achieving 62%.



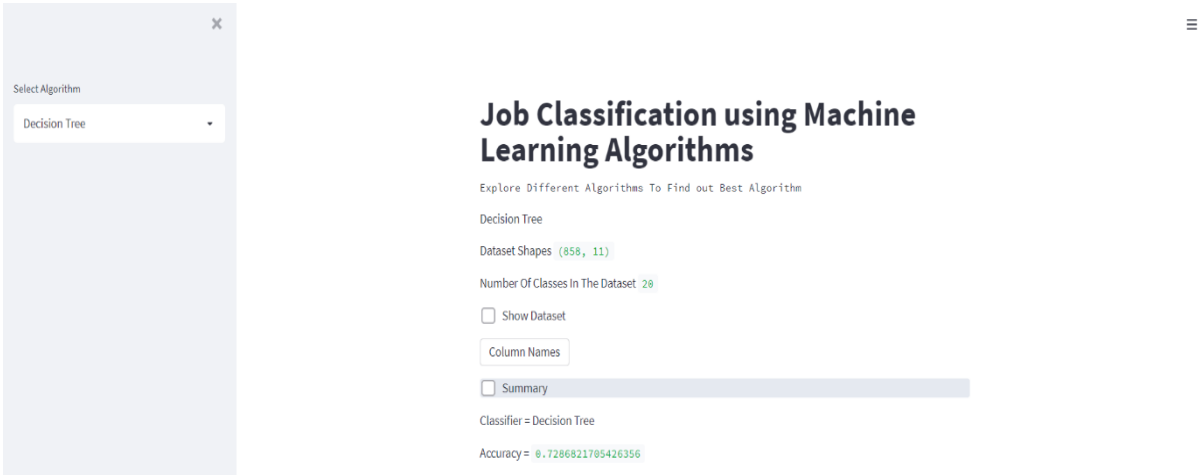
SCREEN 4: Accuracy of Random Forest

The accuracy of the Random Forest is achieving 62%.



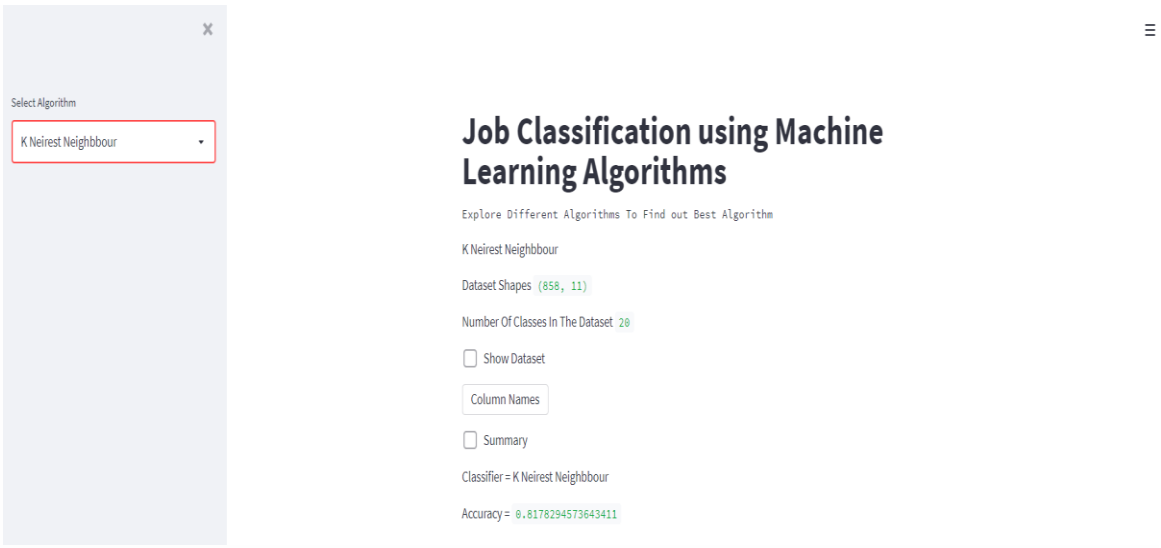
SCREEN 5: Accuracy of Decision tree

The accuracy of the Decision tree is achieving 72%.

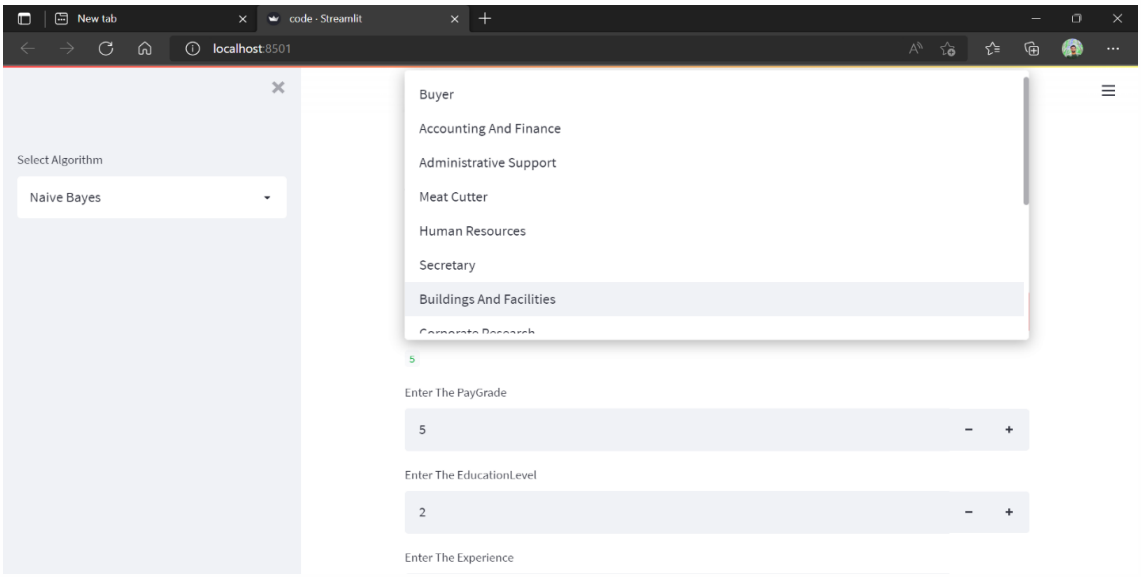


SCREEN 6: Accuracy of KNN:

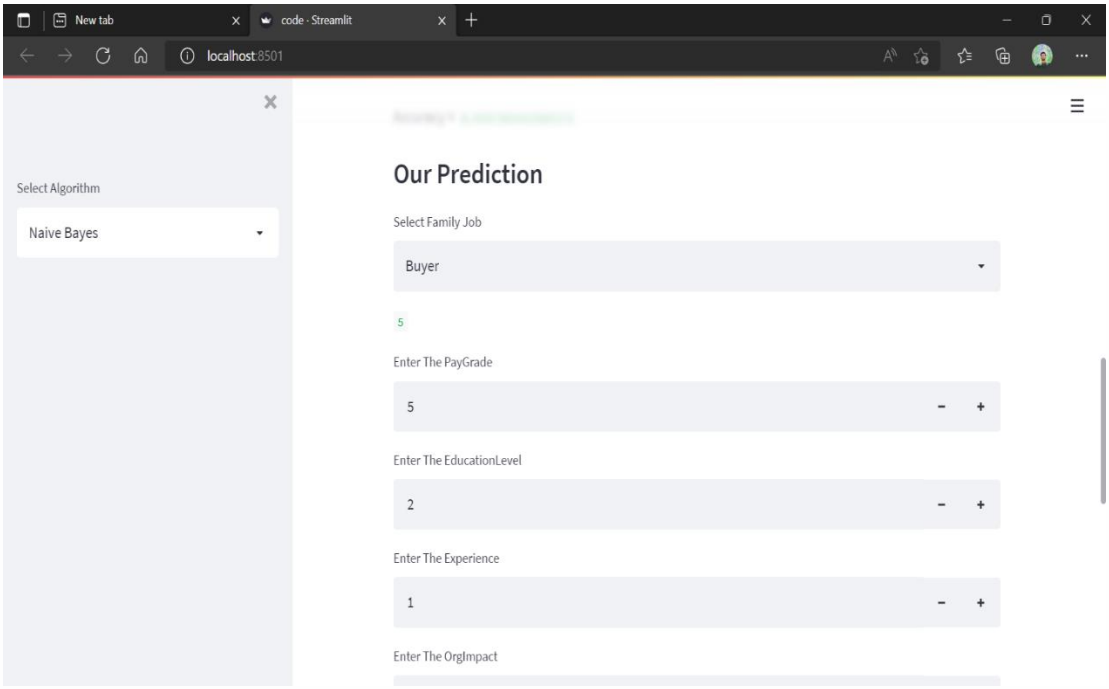
The accuracy of the KNN is about 81.87%.

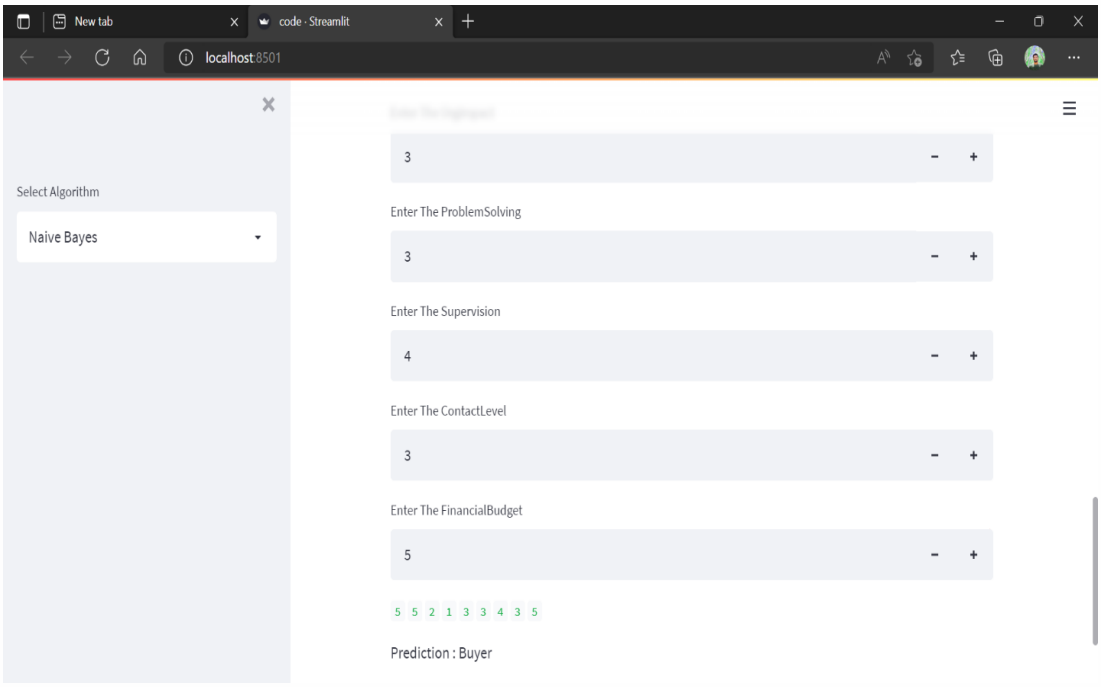


SCREEN 7: Selecting job family



SCREEN 8: Output screens





CHAPTER 5
TESTING & VALIDATION

5.1 INTRODUCTION:

The goal of testing is to improve the program's quality. Quality is assured primarily through some form of software testing. The history of testing goes back to the beginning of the computing field. Testing is done at two Levels of Testing of individual modules and testing the entire system. During the system testing,the system is used experimentally to ensure that the software will run according to the specifications and in the way the user expects. Testing is very tedious and time consuming. Each test case is designed with the intent of finding errors in the way the system will process it.

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the

Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

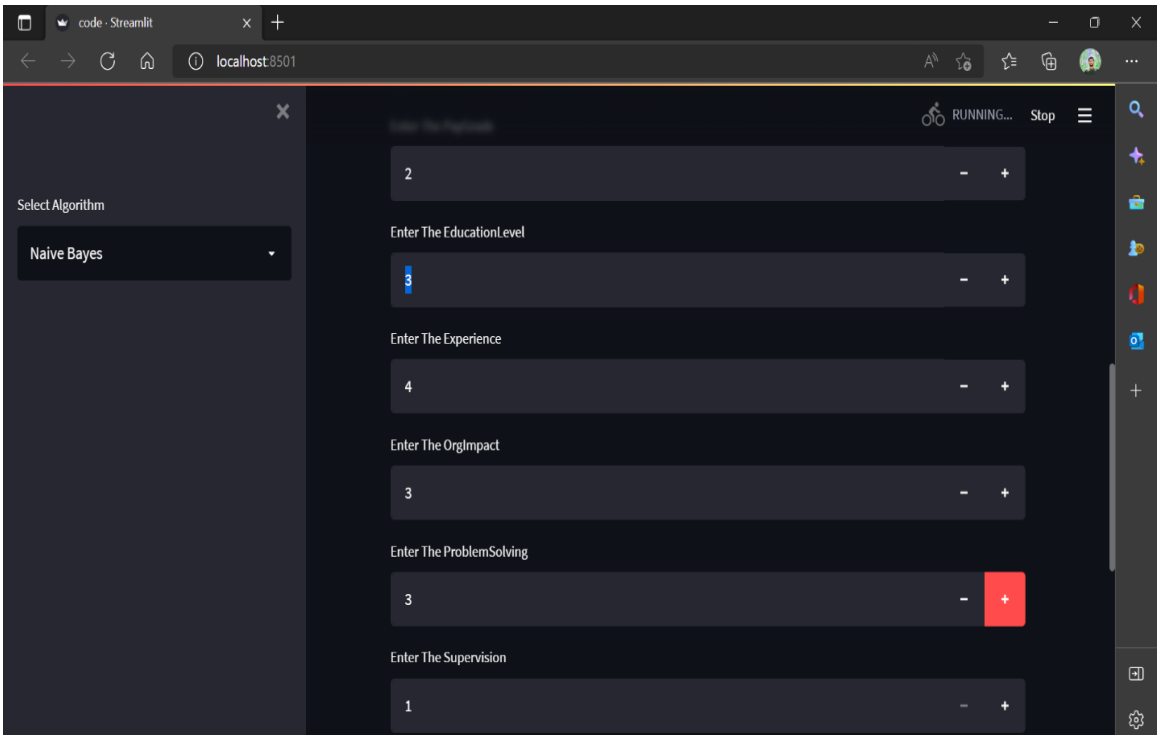
Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

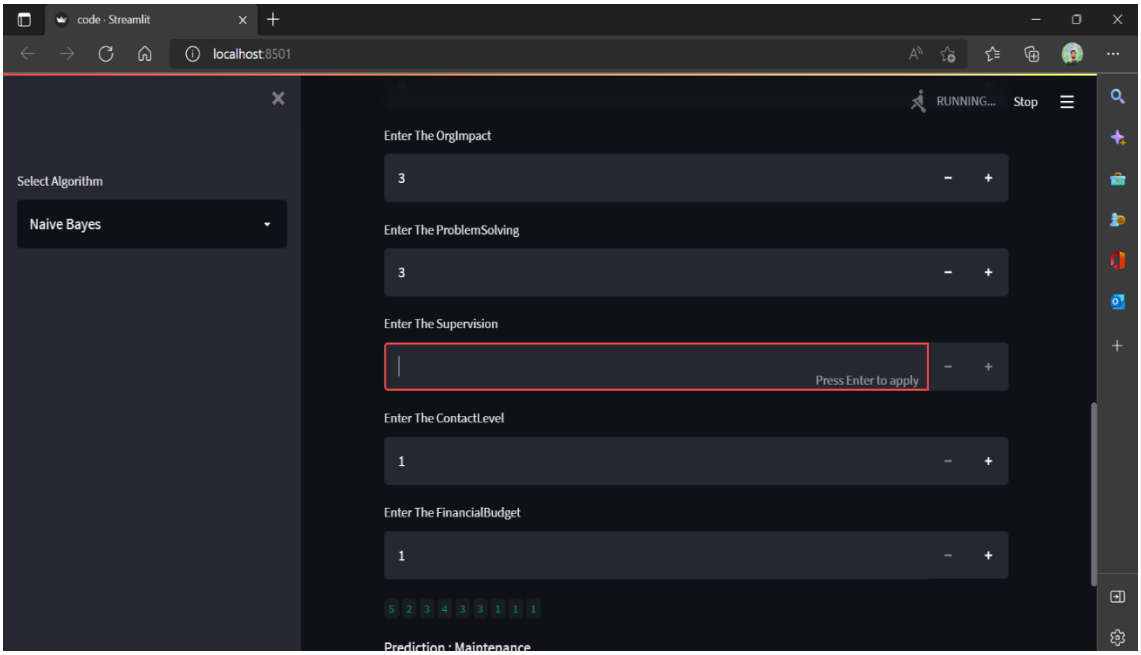
Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.



\

Invalid Input : identified classes of invalid input must be rejected.



Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach:

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing:

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

5.2 DESIGN OF TEST CASES AND SCENARIOS:

Input	Output	Result
Input text	Tested for the classification of student placement data.	Success

5.3 TESTING AND VALIDATION:

Test cases Model building:

S.NO	Test cases	I/P	Expected O/P	Actual O/P	P/F
1	Read the dataset.	Dataset path.	Dataset need to read successfully.	Dataset fetched successfully.	P
2	Performing preprocessing on the dataset	Pre-processing part takes place	Pre-processing should be performed on dataset	Pre-processing successfully completed.	P
3	Model Building	Model Building for the clean data	Need to create model using required algorithms	Model Created Successfully.	P
4	Classification	Input values provided.	Output would be any one of the classifications	Job classified successfully	P

CHAPTER 6

CONCLUSION & FUTURE ENHANCEMENT

CONCLUSION:

This Project uses a rich dataset – to our knowledge, the first dataset with detailed job titles drawn from HR systems from multiple organizations, combined with job descriptions and information about the characteristics of workers – to examine the potential to use machine-learning techniques for occupational classification. We followed the same conceptual framework as that applied by survey methodologists: to define each occupation, to translate concepts to standardized protocols and to build an approach that would infer occupations from the information at hand. Even though the data were drawn from very similar organizations, with very similar production functions, we found that machine-learning approaches were not substantially better than manual classifications. However, we do see the analysis as showing real promise for identifying occupations from job titles combined with a machine-learning approach. The most promising use of the machine learning is that it is an inexpensive way of assigning occupations for job titles that have relatively few people in them and/or for which the algorithm imputes a high degree of accuracy.

Because many job titles have only a few people in them, this approach could yield substantial cost savings (almost 80% of job titles have 10 or fewer people). At the same time, an entirely algorithmic approach would be unwarranted in our case. We also believe that a deeper text analysis of the job descriptions associated with job titles might prove to be a promising approach. Job descriptions typically include information about necessary experience, skills and education which are not only of interest in their own right but could be very useful features for classification purposes. We note that the focus on universities as a subject of analysis has weaknesses and strengths. Major research universities are very large and complicated institutions. There may be other industries in which it might be easier to apply machine learning to job titles. At the same time, the institutions in our sample all come from one narrow sector of the

economy are relatively homogeneous and the data are based on a very specific set of activities (research). We speculate that any classification system for the broader economy would have to be specific to an individual sector or set of sectors.

FUTURE SCOPE:

This can be utilized in future to classify the types of different classifications easily that which can tend to easy to find out the job titles using different machine learning algorithms.

CHAPTER 7

REFERENCES:

1. W. Mellow, H. Sider, Accuracy of response in labor market surveys: Evidence and implications. J. Labor Econ., 331–344 (1983).

2. E. Groshen, Innovating for the Future (2017), (available at https://blogs.bls.gov/blog/tag/bls-products-and-services/).

3. Workforce Information Council Administrative Wage Record Enhancement Study Group, "Enhancing Unemployment Insurance Wage Records Potential Benefits, Barriers, and Opportunities" (Washington DC., 2015), (available at https://www.bls.gov/advisory/bloc/enhancing-unemployment-insurance-wage-records.pdf).

4. N. Zolas et al., Wrapping it up in a person: Examining employment and earnings outcomes for Ph.D. recipients. Science (80-.). 350 (2015), doi:10.1126/science.aac5949.

5. D. Autor, F. Levy, R. MURNANE, The Skill Content of Recent Technological Change: An Empirical Exploration. Q. J. Econ. 118, 4 (2003).

6. K. A. Weeded, D. B. Gru sky, The Case for a New Class Map 1. Am. J. Social. 111, 141 212 (2005).

7. O. Alonso-Villar, C. Del Rio, C. Gradín, The extent of occupational segregation in the United States: Differences by race, ethnicity, and gender. Ind. relations a J. Econ. Soc. 51, 179–212 (2012).

8. D. Acemoglu, U. Akcigit, N. Bloom, W. R. Kerr, "Innovation, reallocation and growth" (National Bureau of Economic Research, 2013).

9. N. G. Peterson et al., Understanding work using the Occupational Information Network (O* NET): Implications for practice and research. Pers. Psychol. 54, 451–492 (2001).

10. C. Levine, L. Salmon, D. H. Weinberg, Revising the standard occupational classification system. Mon. Lab. Rev. 122, 36 (1999).

11. A. Emmel, T. Cosca, the 2010 Standard Occupational Classification (SOC): A Classification System Gets an Update. Occup. Outlook Q. 54, 13–19 (2010).

12. T. Cosca, A. Emmel, Revising the standard occupational classification system for 2010. Mon. Labor Rev. 133, 32–41 (2010).

13. N. A. Mathiowetz, Errors in reports of occupation. Public Opin. Q., 352–355 (1992).

14. J. Bound, C. Brown, N. Mathiowetz, in Handbook of Econometrics (2001; http://www.sciencedirect.com/science/article/pii/S1573441201050127), vol. 5, pp. 3705–3843.

15. K. G. Abraham, J. R. Spletzer, New evidence on the returns to job skills. Am. Econ. Rev. 99, 52–57 (2009).